



UNIVERSITY OF
LEICESTER

**School of Computing and
Mathematical Sciences**

CO7201 Individual Project

Final Report

**A financial decision assistant web
application**

Thomas Shun Hau

Student email: tsh11@student.le.ac.uk

Student ID: 249043010

Project Supervisor: ZHENG, Zedong
Principal Marker: LIU, Daqi

Word Count: 13861
Submission Date: 27 May 2026

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Thomas Shun Hau

Date: 27 May 2026

Abstract

This project designed, implemented and evaluated a web-based financial portfolio assistant for beginner and non-specialist retail investors. The motivation for the project is that individual investors may be able to record or view their investments, but still struggle to interpret whether their portfolio is diversified, aligned with their risk profile, or performing meaningfully against a benchmark. The system was therefore developed as an educational decision-support tool rather than a trading platform or regulated financial adviser.

The application was implemented as a Java Spring Boot web application with a Thymeleaf, Bootstrap and JavaScript frontend, PostgreSQL persistence, Alpha Vantage market-data integration, Chart.js visualisation and Gemini-based natural-language explanation. The system uses an activity-driven portfolio model, where simulated BUY and SELL activities are treated as the source of truth and holdings are derived from them. It supports holdings calculation, allocation analysis, benchmark comparison, portfolio optimisation using Mean-Variance Optimisation and simplified Risk Parity, investor-profile-based suitability analysis, assistant insights, natural-language explanation, onboarding guidance and PDF export.

The system was evaluated using automated unit tests, controller/API tests, integration tests, regression tests, an automated Playwright end-to-end test and IntelliJ coverage inspection. In total, 216 automated tests passed. The evaluation showed that the core assistant and portfolio optimisation packages achieved high coverage, while regression tests addressed practical issues such as missing market prices, market-data quota risks and optimisation eligibility. The final system achieved its main aim by combining portfolio tracking, analysis, optimisation and explanation into a coherent educational workflow.

Contents

Abstract	3
1. Introduction	8
1.1. Background and motivation.....	8
1.2. Problem statement.....	9
1.3. Structure of the report	9
2. Aims and Objectives	10
2.1. Aims.....	10
2.2. Objectives	10
2.3. Requirements.....	11
2.3.1. Essential Requirements	11
2.3.2. Recommended Requirements	12
2.3.3. Optional Requirements	13
3. Background and Literature Review	14
3.1. Existing Products and Related Systems	14
3.2. Retail Investor Challenges and Educational Decision Support	15
3.3. Financial Concepts Used in the System	15
3.4. Portfolio Optimisation and Performance Measurement.....	16
3.5. Benchmark Comparison	17
3.6. Attention-Based Transformers	17
3.7. Large Language Models for Financial Explanation	18
3.8. Justification of Background Research.....	18
3.9. Summary.....	18
4. System architecture and design	19
4.1 Architectural Overview	19
4.2 Technology Stack and Design Rationale	19
4.3. Frontend and Onboarding Design.....	20
4.4. Backend Architecture Overview	20
4.5. Web and Security Layer Design	21
4.6. Portfolio Services Design.....	22
4.7. Financial Assistant Services Design	22
4.8 External Integration Design	24
4.9 Data Model, Persistence and Repository Design.....	24
4.10 Activity-Driven Portfolio Workflow	25
4.11 Portfolio Optimisation Design.....	27

4.12 Explainability Design.....	29
4.13 PDF Export Design.....	29
4.14 Error Handling and Robustness.....	29
4.15 Summary	30
5. Implementation and Development Process.....	31
5.1 Implementation Overview	31
5.1.1 Code Organisation	31
5.1.2 Development Process	32
5.1.3 Use of Generative AI in Implementation.....	32
5.1.4 Key Implementation Challenges.....	32
5.2 Backend Implementation	32
5.2.1 Authentication and User-Specific Data.....	32
5.2.2 Investment Activity Management	32
5.2.3 Holdings Calculation from Activities	33
5.2.4 Asset Profiles and Metadata Synchronisation	33
5.2.5 Allocation Analysis	33
5.2.6 Benchmark Comparison.....	34
5.2.7 Portfolio Optimisation	34
5.2.8 Financial Consultant Assistant	35
5.2.9 Natural-Language Explanation.....	36
5.2.10 Caching and API Quota Handling	36
5.2.11 Error Handling and Warnings	36
5.3 Frontend Implementation.....	36
5.3.1 Frontend Code Organisation	36
5.3.2 Shared Portfolio Header and Navigation	37
5.3.3 New-User Onboarding Flow	37
5.3.4 Investor Profile Page and Help Popovers	38
5.3.5 Activities, Holdings and Allocation Pages	38
5.3.6 Analysis Page: Benchmark, Optimisation and Explanation	39
5.3.7 PDF Export.....	42
5.4 Summary	42
6. Testing and Evaluation.....	42
6.1 Testing Strategy.....	42
6.2 Test Code Organisation.....	43
6.3 Requirements Evaluation Plan.....	44
6.3.1 Essential Requirements	44
6.3.2 Recommended Requirements.....	45

6.3.3 Optional Requirements	45
6.4 Automated Unit Testing	46
6.4.1 Holdings Calculation Tests.....	46
6.4.2 Allocation Analysis Tests	46
6.4.3 Financial Assistant Tests	46
6.4.4 Portfolio Optimisation Tests	46
6.5 Controller and API Testing.....	47
6.6 Integration and Regression Testing	47
6.7 Automated End-to-End Testing.....	47
6.8 Coverage and Test Result Summary.....	48
6.9 Evaluation of Generative AI-Assisted Component.....	48
6.10 Limitations of Testing	49
6.11 Summary	49
7. Results and Discussion	49
7.1 Overview.....	49
7.2 Achievement of Project Aim and Requirements.....	49
7.3 Discussion of Main System Results.....	50
7.3.1 Activity-Driven Portfolio Workflow	50
7.3.2 Portfolio Analysis and Visualisation.....	50
7.3.3 Benchmark Comparison and Optimisation	50
7.3.4 Financial Consultant Assistant	51
7.3.5 Natural-Language Explanation and PDF Export	51
7.4 Comparison with Existing Systems and Background Findings	51
7.5 Strengths of the Final System.....	52
7.6 Challenges and Resolutions	52
7.7 Limitations	52
7.8 Summary	53
8. Conclusion and Future Work	53
8.1 Project Conclusion.....	53
8.2 Future Work	53
8.2.1 Self-Developed or Fine-Tuned Explanation Model.....	54
8.2.2 Comparison of Different LLM or NLP Models.....	54
8.2.3 Feedback Page and User Evaluation.....	54
8.2.4 Other Technical Extensions	54
8.3 Final Summary.....	54
Reference.....	55
Appendices	56

A. Database Schema.....	56
B. Sample Exported PDF Report.....	58
C. IntelliJ Coverage Report	62

1. Introduction

Financial investment has become increasingly accessible to individual retail investors through online brokerages, investment platforms, and financial information services. These investors have access to market transactions, price movements, and other market data with ease, but most of them do not have enough technical knowledge to analyse and make use of the information. Although novice investors may know which financial assets they hold, they often lack the knowledge to judge whether their portfolio is diversified, overly concentrated, aligned with their investment goals, or performing better or worse than market benchmarks.

This project addresses this problem by designing and implementing a financial portfolio assistant web application for beginner and intermediate retail investors. The system is intended to help users record simulated investment activities, derive portfolio holdings, analyse allocation, compare performance against market benchmarks, calculate portfolio optimisation scenarios, and receive beginner-friendly explanations of the results. The project is designed as a decision support tool that helps users better understand portfolio information and the implications of calculated analysis results. However, it does not execute real trades.

1.1. Background and motivation

Existing investment platforms and online financial services have made it possible for individual investors to access market data, monitor prices, and manage investment portfolios. However, access to financial data does not automatically mean that users can interpret it correctly. Beginner investors may be able to see the value of their holdings, daily price changes, and unrealised gains or losses, but may still struggle to understand the wider meaning of their portfolio structure.

A portfolio is a collection of financial assets. Its usefulness depends on how those assets interact with each other, how much risk the user is exposed to, whether the portfolio is diversified, and whether its performance is meaningful when compared with an appropriate benchmark. Concepts such as asset allocation, volatility, return, Sharpe ratio, and portfolio optimisation are important in investment analysis, but they can be difficult for users without a finance background to understand.

Existing portfolio tracking systems tend to focus on recording transactions and displaying portfolio values. But they do not fully address the interpretability problem faced by non-specialist users. A beginner investor may still ask whether their portfolio is too heavily concentrated in one asset class, whether their current allocation matches their risk tolerance, or why an optimisation result suggests changing the allocation. Therefore, there is a need for finance systems that not only present data, but also help users interpret the data.

This project responds to that need by implementing a portfolio assistant rather than a passive portfolio dashboard. The system follows an activity-driven design, where user investment activities are used as the source of truth. From these activities, the system derives holdings, calculates allocation, compares performance against benchmarks, calculates optimised portfolio scenarios using the selected portfolio optimisation algorithm, and produces explanatory outputs. The project also includes an investor-profile-based assistant layer, where user information such as investment goal, horizon, experience, and risk tolerance is used to calculate suitability insights.

Another motivation is transparency. The project aims to make portfolio calculations more understandable by combining visualisation, warnings, suggested changes and natural-language explanation. The LLM component explains results calculated by backend services rather than independently calculating financial recommendations.

1.2. Problem statement

The main problem addressed by this project is that beginner and non-specialist investors may be able to record or view their investments but may struggle to interpret whether their portfolio is suitable, diversified, or performing meaningfully. A traditional portfolio tracker can show holdings, values, and gains or losses, but they do not answer more interpretive questions such as:

- Is the portfolio too concentrated in one asset class, sector, market, or currency?
- Is the portfolio performing better or worse than a relevant benchmark?
- What does volatility or Sharpe ratio mean in the context of the user's portfolio?
- Why does the selected optimisation algorithm suggest a different allocation?
- Does the current allocation match the user's investment horizon, goal, and risk tolerance?

This project therefore addresses both the calculation problem and the explanation problem. The calculation problem is to derive accurate portfolio data from user activities, integrate market data, calculate allocation, compare benchmark performance, and calculate optimisation results. The explanation problem is to present those results in a form that is understandable to users without specialist finance knowledge.

The system solves this through a full-stack web application that combines activity-driven portfolio tracking, external market data integration, allocation analysis, benchmark comparison, portfolio optimisation using portfolio optimisation algorithms, beginner-friendly explanation, PDF export, and investor-profile-based suitability insights. The system is designed to support user understanding, not to provide regulated financial advice or guarantee investment outcomes.

1.3. Structure of the report

Chapter 2 presents the aims, objectives, and requirements of the project. It explains the essential, recommended, and optional requirements. These requirements are addressed in later chapters.

Chapter 3 provides the background research and literature review. It discusses existing products and related systems, retail investor challenges, portfolio diversification, asset allocation, portfolio optimisation, benchmark comparison, attention-based transformers, and Large Language Models for financial explanation.

Chapter 4 describes the system architecture and design. It explains the full-stack architecture, activity-driven portfolio model, data model, backend and frontend design, market data integration, portfolio analysis design, Financial Consultant Assistant design, explainability design, PDF export design, and error-handling approach.

Chapter 5 explains the implementation and development process. It describes the main implemented features, including authentication, activity management, holdings calculation, asset profiles, allocation analysis, overview dashboard, benchmark comparison, portfolio optimisation, warnings and exclusions, explanation system, PDF export, Financial Consultant Assistant, caching, API quota handling, and global error handling.

Chapter 6 presents the testing and evaluation of the system. It discusses the testing strategy, black-box tests, white-box tests, controller and API tests, integration tests, regression tests, automated Playwright end-to-end testing, coverage results, PDF export evaluation, API quota-related regression testing, and evaluation against the project requirements.

Chapter 7 discusses the results of the project. It evaluates how well the project aim was achieved, how the system supports beginner investors, and the strengths and limitations of the project.

Chapter 8 concludes the report and presents future work. It summarises the final system and identifies possible additions to improve the system.

2. Aims and Objectives

2.1. Aims

The aim of this project is to design, implement, and evaluate a web-based financial portfolio assistant for beginner and non-specialist retail investors. The system is intended to help users record simulated investment activity, understand the structure and risk exposure of their portfolio, compare performance against benchmarks, calculate optimisation scenarios, and receive educational explanations of technical results.

The system is designed as an active assistant. It displays investment records and market values, derives holdings from user activities, compares the current portfolio against a user-specific investor profile, analyses portfolio allocation, calculates benchmark and optimisation results, and explains those results in beginner-friendly language. This reflects the project's final design position that the system should act as a portfolio assistant rather than a passive dashboard.

It is important to note that this application is intended solely as a decision support and educational tool. It does not perform real trading activities and does not guarantee financial returns or investment performance.

2.2. Objectives

To achieve the aim, the project was divided into the following objectives:

1. Implement user authentication and user-specific portfolio data management.

Users should be able to register, log in, and access portfolio data linked to their own account.

2. Implement an activity-driven portfolio model.

Users should be able to record simulated BUY and SELL activities. These activities should act as the source of truth from which holdings, allocation, analysis results, assistant insights, and exported reports are derived.

3. Calculate portfolio holdings from activity records.

The system should aggregate user activities to calculate total quantity, total cost, average cost, latest price, market value, unrealised gain/loss, and unrealised gain/loss percentage.

4. Integrate external market data.

The system should retrieve supported asset search results, metadata, latest prices, historical prices, and benchmark/index data from an external market data provider.

5. Implement asset profile management and metadata synchronisation.

The system should store asset metadata such as display name, asset class, sector, market, currency, data source, and support flags for price data and optimisation.

6. Implement investor-profile-based assistant functionality.

The system should collect user profile information such as age, investment horizon, investment goal, risk tolerance, investment experience, and income stability. It should use this information to calculate a risk profile, target allocation, suitability comparison, and assistant insights.

7. Provide portfolio allocation analysis.

The system should calculate and display allocation by holding, asset class, sector, market, and currency to help users understand diversification and concentration.

8. Implement benchmark comparison.

The system should compare portfolio performance against a selected benchmark or index and display portfolio return, benchmark return, excess return, and indexed performance visualisation.

9. Calculate portfolio optimisation using portfolio optimisation algorithm.

The system should use historical price data, returns, covariance, current weights, and constraints to calculate optimisation results, including expected annual return, annual volatility, Sharpe ratio, efficient frontier points, target allocation, suggested changes, and warnings.

10. Provide beginner-friendly explanation of analysis results.

The system should use an explanation service to convert calculated portfolio results into readable text for users without specialist finance knowledge.

11. Implement PDF export of portfolio analysis.

The system should allow users to export the analysis page as a reusable PDF report containing available portfolio summary, holdings, allocation, benchmark comparison, optimisation results, warnings, suggestions, and explanations.

12. Evaluate the system using structured testing methods.

The system should be tested using unit tests, controller/API tests, integration tests, regression tests, failure-handling tests, performance/quota regression tests, and automated end-to-end testing.

2.3. Requirements

The requirements are grouped into essential, recommended, and optional requirements.

2.3.1. Essential Requirements

- User authentication

The system must allow users to register, log in, and access only their own portfolio data.

- Investment activity management

The system must allow users to create, view, update, and delete investment activities.

- Derive finance holdings from activities

The system must derive holdings and their metrics from BUY and SELL activities rather than requiring users to manually maintain holdings.

- Calculate holding metrics

The system must calculate quantity, total cost, average cost, market value, and unrealised gain/loss where data is available.

- Market data integration

The system must integrate with an external market data provider for supported asset search, latest prices, historical prices, and benchmark data.

- Asset allocation visualisation

The system must calculate and display allocation by holdings, asset class, sector, market, and currency.

- Benchmark comparison

The system must compare current user portfolio with a selected benchmark/index and show return performance metrics and comparison results.

- Portfolio optimisation using portfolio optimisation algorithm

The system must calculate optimised portfolio results using supported historical market data.

- User Interface

The system must provide web pages for portfolio overview, activities, holdings, asset profiles, allocation, analysis, and investor profile.

- Error handling and warnings

The system must provide clear error responses and user-facing warnings when input is invalid, external market data is unavailable, or assets cannot be included in optimisation.

2.3.2. Recommended Requirements

- Natural-language explanation using an existing model

The system should provide beginner-friendly explanations of calculated portfolio analysis and optimisation results using an existing Large Language Model service. The explanation component should only explain results calculated by deterministic backend services and should not independently calculate investment recommendations.

- Investor profile

The system should allow users to complete an investor profile containing age, investment horizon, investment goal, risk tolerance, investment experience, and income stability.

- Risk assessment

The system should classify the user into a risk profile, such as Conservative, Balanced, Growth, or Aggressive, based on the investor profile.

- Target allocation recommendation

The system should calculate a target allocation based on the user's risk profile.

- User allocation comparison against target allocation

The system should compare the user's actual portfolio allocation with the calculated target allocation and identify major differences.

- Assistant insights

The system should display assistant insights on the overview page to help the user understand whether their portfolio appears aligned with their profile.

- API quota reduction and caching

The system should reduce unnecessary external market-data API calls by using batching, caching, and avoiding automatic expensive requests where possible.

- New-user onboarding

The system should guide new users through the initial setup process by encouraging completion of the investor profile and creation of the first activity. Navigation should indicate which analysis pages become useful after setup is complete.

2.3.3. Optional Requirements

- PDF export

The system may allow users to export the available portfolio analysis results into a PDF report for later review.

- Multiple optimisation algorithms

The system may support additional portfolio optimisation algorithms.

- Natural-language explanation using a self-developed model

The system may use a self-developed or fine-tuned natural-language model instead of relying on an external LLM service. This was not implemented in the final project due to time constraints and because the project focus was on integrating explanation into the portfolio assistant workflow.

- Comparison of different NLP or LLM models

The system may compare different language models for explanation quality, readability, accuracy, and consistency. This remains future work.

- Feedback page

The system may allow users to provide feedback on explanations, assistant insights, and usability. This could support future evaluation and refinement.

3. Background and Literature Review

This chapter reviews the background knowledge and existing work relevant to the project.

3.1. Existing Products and Related Systems

Existing financial software systems can be grouped into portfolio trackers, robo-advisors, financial analytics platforms and AI-assisted finance tools. Portfolio trackers such as Ghostfolio are designed mainly to help users record assets, monitor holdings, view gains or losses and produce performance reports [14]. Their strength is that they organise portfolio data and often provide allocation or performance visualisation. However, they usually focus on presenting data rather than explaining how a beginner investor should interpret the data. The use of clear visualisation is still important, because well-designed displays can make quantitative information easier to compare and interpret [7].

Robo-advisors and digital wealth management platforms provide a more active form of investment support. Services such as Moneyfarm collect information such as risk tolerance, investment horizon and investment goals, and then use that information to recommend or manage a portfolio [13]. Risk tolerance research also shows why investor profiling matters, because demographic information alone is not enough to describe how much investment risk a user is willing or able to accept [20]. This shows that investor profiling and target allocation are common ideas in digital investment systems. However, many robo-advisor platforms are designed for real investment management and do not necessarily expose the reasoning process in a way that is educational for beginner users.

Financial analytics platforms, such as OpenBB and similar market-data tools, provide access to market data, indicators, company information and research workflows [15]. These systems are designed for users with finance knowledge, but the volume of information can be difficult for non-specialist users to interpret. AI-assisted systems, including finance-specific language models and financial agent frameworks, show growing interest in combining structured financial data with natural-language explanation [10, 11].

The project is positioned between these categories. It is not a full robo-advisor because it does not manage real money or execute trades. It is also more than a passive portfolio tracker because it includes allocation analysis, benchmark comparison, optimisation and explanatory support. It is also more than just a financial analytics platform as it provides explanation to finance results. Table 3.1 summarises the relationship between existing system categories and this project.

Feature	Portfolio trackers	Robo-advisors	Financial analytics platforms	This project
Activity tracking	Yes	Depends on product	Depends on product	Yes
Derived holdings	Yes	Yes	Depends on product	Yes
Allocation analysis	Yes	Yes	Yes	Yes
Benchmark comparison	Depends on product	Depends on product	Yes	Yes

Portfolio optimisation	No	Yes	Limited	Yes
Investor profile	Uncommon	Common	Uncommon	Yes
Beginner-friendly explanation	Depends on product	Yes	Depends on product	Yes
LLM-based explanation	Uncommon	Uncommon	Uncommon	Yes

Table 3.1. Comparison of this project against existing market products and projects.

3.2. Retail Investor Challenges and Educational Decision Support

Retail investing has become more accessible through online investment platforms, but access to financial data does not guarantee understanding. Kuzniak, S. et al. in their studies of retail investor behaviour and risk tolerance suggest that investors can struggle to interpret financial information and that perceived knowledge may differ from actual decision-making competence [20]. A beginner investor may know the names and values of the assets they own, but still struggle to judge whether the portfolio is diversified, too risky, or aligned with their investment goals. This creates a gap between portfolio information and portfolio interpretation.

One difficulty is that many finance concepts are expressed using specialist terms. Volatility, covariance, Sharpe ratio, benchmark return and efficient frontier are useful concepts in portfolio analysis, but they may not be meaningful to a non-specialist unless supported by explanation [20]. A second difficulty is that performance is relative. A portfolio can have a positive return but still underperform a relevant benchmark, or it can show a gain while being concentrated in one asset class, sector or market [21].

Educational decision-support systems are relevant because they help users interpret information without taking final control away from them. In this project, decision support means presenting calculations, visualisations, warnings and explanations so that users can better understand their portfolio. The system therefore supports users in interpretation and learning through transparent calculations and explanatory text.

3.3. Financial Concepts Used in the System

Several finance terms need to be defined because they are used throughout the system design and implementation. A portfolio is a collection of financial assets held by a user. In this project, a holding is the derived quantity and value of a particular asset currently owned by the user. An activity is a simulated investment event, such as a BUY or SELL transaction.[1] This distinction is important because the system treats activities as the source of truth and derives holdings from them.

Asset allocation refers to how the total portfolio value is distributed across categories such as asset class, sector, market and currency [1,2]. Diversification refers to spreading exposure across multiple assets or categories to reduce dependence on a single asset or source of risk. Markowitz's portfolio theory formalised the importance of diversification by showing that portfolio risk depends on covariance between assets, not only on the risk of each asset individually [1]. Diversification can reduce asset-specific risk, but it cannot remove all market risk. These are the underlying concepts that lead to the implementation of allocation analysis functions of the system.

Return is the gain or loss on an investment over a period, usually expressed as a percentage. Volatility measures how much returns vary over time and is often used as a proxy for risk. Covariance measures how two assets move in relation to each other,

which is important because portfolio risk depends not only on each asset individually, but also on how assets interact [1]. The Sharpe ratio is a risk-adjusted performance measure that compares return above a risk-free rate with volatility [3].

A benchmark is a reference index or asset used to compare performance. Benchmark comparison helps users interpret whether a portfolio return is meaningful relative to a wider market reference. Performance evaluation literature treats portfolio performance as a relative concept, where returns are compared against an appropriate benchmark rather than judged in isolation [21]. A risk profile is a simplified classification of the user's willingness and ability to take investment risk. A target allocation is a reference allocation associated with a risk profile [22]. In this project, these terms are used educationally to support suitability comparison rather than regulated advice.

3.4. Portfolio Optimisation and Performance Measurement

Portfolio optimisation is one of the main analytical concepts underlying the project. The first implemented optimisation approach is based on Mean-Variance Optimisation, which originates from Modern Portfolio Theory. Markowitz introduced the idea that portfolio selection should consider both expected return and risk, where risk can be represented using variance or standard deviation of returns [1].

The central idea is that a portfolio should not be assessed only by the expected return of each individual asset. The relationship between assets also matters. If assets do not move together perfectly, combining them can reduce overall portfolio volatility. This is why covariance and diversification are important in portfolio optimisation.

Mean-Variance Optimisation is commonly represented through the efficient frontier. The efficient frontier describes portfolios that offer the highest expected return for a given level of risk, or the lowest risk for a given level of return [1]. In a software system, this requires historical price data, return calculation, covariance estimation and constraints such as long-only weights or maximum allocation limits. The project uses this theory to calculate optimisation scenarios, but the results are presented as educational calculations rather than predictions of future performance.

The expected return of a portfolio can be written as:

$$E(Rp) = \sum w_i \mu_i, \text{ for } i = 1 \text{ to } n$$

where w_i is the weight of asset i and μ_i is the expected return of asset i .

The portfolio variance can be written as:

$$\sigma_p^2 = \sum \sum w_i w_j \sigma_{ij}, \text{ for } i = 1 \text{ to } n \text{ and } j = 1 \text{ to } n$$

where σ_{ij} is the covariance between asset i and asset j . This shows that portfolio risk depends not only on each asset's individual volatility, but also on the covariance between assets.

In a long-only portfolio, the weights are constrained so that:

$$\sum w_i = 1$$

$$w_i \geq 0$$

The approach also has limitations. Mean-Variance Optimisation is sensitive to input assumptions, especially expected return estimates and covariance. Small changes in historical data or assumptions may produce noticeably different allocation weights [4]. This limitation is important for the project because beginner users may over-trust

calculated output if the system does not explain its assumptions. Therefore, optimisation results need to be accompanied by warnings and explanations.

To provide a contrasting optimisation approach, a second portfolio optimisation method, simplified Risk Parity, is also implemented. Risk Parity focuses on balancing risk exposure rather than directly maximising expected return. In a full risk parity model, the objective is commonly to allocate capital so that assets contribute more equally to total portfolio risk. A simplified version of this idea is inverse-volatility weighting, where assets with higher historical volatility receive lower target weights and assets with lower historical volatility receive higher target weights. This project uses the simplified inverse-volatility approach as an interpretable risk-focused alternative to Mean-Variance Optimisation [5, 6].

The annualised volatility of asset i is calculated from its daily return series:

$$\sigma_i = \sqrt{252} \times \text{standard deviation of daily returns for asset } i$$

The inverse-volatility score for asset i is then:

$$a_i = \frac{1}{\sigma_i}$$

The target weight for asset i is calculated by normalising the inverse-volatility scores:

$$w_i = a_i / \sum a_k, \text{ for } k = 1 \text{ to } n$$

This means that an asset with lower historical volatility receives a larger target allocation, while an asset with higher historical volatility receives a smaller target allocation.

Simplified Risk Parity also has limitations. It relies on historical volatility and assumes that past volatility is useful for estimating future risk. The implemented version does not calculate marginal risk contribution or solve a full equal-risk-contribution optimisation problem. It should therefore be described as simplified Risk Parity or inverse-volatility weighting rather than full institutional Risk Parity.

3.5. Benchmark Comparison

Benchmark comparison is used to interpret portfolio performance against a reference. Portfolio performance evaluation is commonly framed as a relative assessment, because benchmark-relative comparison helps distinguish general market movement from portfolio-specific performance [21]. A portfolio return may appear good when viewed alone, but the interpretation changes if a relevant benchmark performed much better or worse over the same period. For example, a positive portfolio return can still represent underperformance if the selected benchmark increased by a larger amount.

The project uses benchmark comparison as a supporting analysis rather than a complete judgement of portfolio quality. A portfolio containing several asset classes, currencies or markets may not be fully represented by a single equity index. For this reason, benchmark comparison is only one of many analytical methods used in this project.

3.6. Attention-Based Transformers

The transformer introduced attention mechanisms that allow a model to consider relationships between different parts of an input sequence when producing output [8].

This is relevant because portfolio explanation requires the model to use several pieces of structured context together.

3.7. Large Language Models for Financial Explanation

Finance-oriented models and agent systems show that there is growing interest in combining structured financial data with natural-language reporting [10, 11].

Data-to-text generation converts structured information, such as tables or metrics, into natural-language text [9].

The use of an LLM also introduces risks. Language models can produce fluent text that is incomplete, inconsistent or unsupported by the input if not constrained, and hallucination is a recognised problem in LLM-based systems [23]. This is especially important in a finance-related application. The final design of this project therefore separates calculation from explanation. Backend services calculate the portfolio metrics then pass the calculated results to LLM to have it perform data-to-text transformation.

NLG evaluation often requires human or task-specific assessment rather than only automatic metrics [12].

3.8. Justification of Background Research

The background research was selected to support the main design decisions of the project. Existing portfolio trackers, robo-advisors and financial analytics platforms were reviewed to understand how current systems record portfolio data, present analysis and support investment decision-making. This comparison showed that many systems provide portfolio tracking or financial data access, but fewer focus on explaining calculated results to beginner users in an educational way. This finding supported the decision to design the system as a portfolio assistant rather than only a passive dashboard.

The finance background was necessary because the system implements portfolio analysis functions rather than only displaying stored records. Concepts such as holdings, allocation, diversification, benchmark comparison, volatility, Sharpe ratio and optimisation are used throughout the design and implementation. Reviewing these concepts helped define what the system should calculate and also clarified the limits of the system. For example, Mean-Variance Optimisation and simplified Risk Parity can calculate educational allocation scenarios, but they should not be presented as predictions of future performance or regulated financial advice.

The review of transformers, Large Language Models and data-to-text generation was included because the project uses an LLM to explain calculated portfolio results. The main conclusion from this investigation was that an LLM is suitable for transforming structured analysis results into readable explanation, but should not be responsible for calculating financial metrics. This directly influenced the final design: backend services calculate holdings, allocation, benchmark comparison, optimisation outputs and assistant insights, while the LLM is used only as an explanation layer.

3.9. Summary

This chapter reviewed the existing systems and background concepts that support the project. Portfolio trackers, robo-advisors, financial analytics platforms and financial AI systems each address part of the problem, but the project combines selected ideas from these categories into an educational portfolio assistant. The finance concepts explain why holdings, allocation, diversification, benchmark comparison, risk-adjusted performance and Mean-Variance Optimisation and simplified Risk Parity are relevant to the system. It also explains why transformer-based LLMs are suitable for explaining calculated results. These ideas form the basis for the design decisions to be discussed in Chapter 4.

4. System architecture and design

This chapter describes the architecture and design of the financial portfolio assistant web application. The core design philosophy is to support a clear separation between user interface logic, portfolio calculation, financial assistant logic, market-data integration, persistence, and natural-language explanation. This architectural design ensures that core business logic remains in the backend, while external APIs are managed through integration layers.

The system follows a layered web application structure. The frontend is responsible for presenting pages, sending asynchronous requests, rendering charts, and exporting available analysis data to PDF. The backend is responsible for authentication, user-specific data access, activity management, holdings calculation, allocation analysis, benchmark comparison, portfolio optimisation, investor-profile suitability analysis, explanation preparation, and error handling. Persistent data is stored in PostgreSQL, while external market data is retrieved from Alpha Vantage and explanation requests are sent to Gemini through Google GenAI.

The design uses two important principles. First, investment activities are treated as the source of truth. Holdings are not manually stored as independent records; instead, they are derived from BUY and SELL activities. This reduces inconsistency between recorded transactions and calculated portfolio state. Second, the Large Language Model is used only as an explanation layer. Portfolio metrics and recommendations are calculated by backend services before being passed to the explanation component. This keeps financial calculation deterministic and testable.

4.1 Architectural Overview

The overall system is a full-stack Spring Boot web application. The user interacts with the system through a browser-based interface built from Thymeleaf templates, Bootstrap styling, JavaScript, AJAX requests, and Chart.js visualisations. The browser sends normal page requests for server-rendered pages and AJAX requests for dynamic portfolio data. The analysis page also contains a frontend PDF export script using jsPDF and jspdf-autotable.

The Spring Boot backend contains the main application logic. Controllers handle page and JSON requests, services contain portfolio and assistant logic, repositories access PostgreSQL, and integration components isolate Alpha Vantage and Gemini.

Figure 4.1: Overall System Architecture

4.2 Technology Stack and Design Rationale

The backend was implemented using Java and Spring Boot because it supports structured web applications, REST APIs, dependency injection, validation, security and data access [17].

PostgreSQL was used as the final database for users, investment activities, asset profiles and investor profiles [24]. Derived outputs such as holdings, allocation summaries, optimisation results and assistant insights are calculated when required rather than stored as separate tables.

The frontend uses Thymeleaf, Bootstrap, JavaScript/AJAX and Chart.js for server-rendered pages and dynamic visualisation [18]. PDF export is implemented in the frontend using jsPDF and jspdf-autotable.

Alpha Vantage was chosen as the market-data provider for asset search, metadata, latest prices, historical prices and benchmark data [16]. Caching and request reduction were included because market-data APIs can be rate-limited.

The explanation feature uses Gemini through Google GenAI [25]. Backend services calculate the financial results first, and the LLM converts those results into explanatory text.

4.3. Frontend and Onboarding Design

The frontend design focuses on making the portfolio workflow understandable for beginner users. Since the system contains several portfolio pages, including overview, investor profile, activities, asset profiles, holdings, allocation, and analysis, the interface needs to guide users through the intended order of use without making the application difficult to navigate.

A shared portfolio header is used across the main portfolio pages to provide consistent navigation, user greeting and logout behaviour.

The frontend also includes onboarding support for new users. The intended setup flow is:

Overview → Investor Profile → Activities → Full portfolio analysis pages
--

Setup is considered complete when the user has both an investor profile and at least one recorded activity. Before this, analysis-focused pages are visually disabled as guidance rather than as a security restriction.

The overview page includes an onboarding popup that encourages new users to complete their investor profile first.

The investor profile page includes help popovers beside questionnaire fields so that beginner users can understand terms such as risk tolerance, investment experience and investment goal.

Figure 4.2: Frontend Component Design

4.4. Backend Architecture Overview

The backend is organised into several large components: security layer, web layer, service layer, portfolio optimisation module, financial assistant module, external integration layer, repository layer, and global error handling. This structure supports separation of concerns and makes the system easier to test.

The security layer is responsible for authentication and user-specific access. It authenticates users, creates an application-level user principal, and makes the current user identity available to backend components. This is important because portfolio data must be linked to the correct account. Controllers and services should operate on the authenticated user's data rather than exposing records globally.

At a high level, the web layer handles page and JSON requests, the service layer contains business logic, the repository layer accesses persisted entities, and the integration layer isolates Alpha Vantage and Gemini. Later sections describe these areas in more detail.

Figure 4.3 Backend Architecture Overview

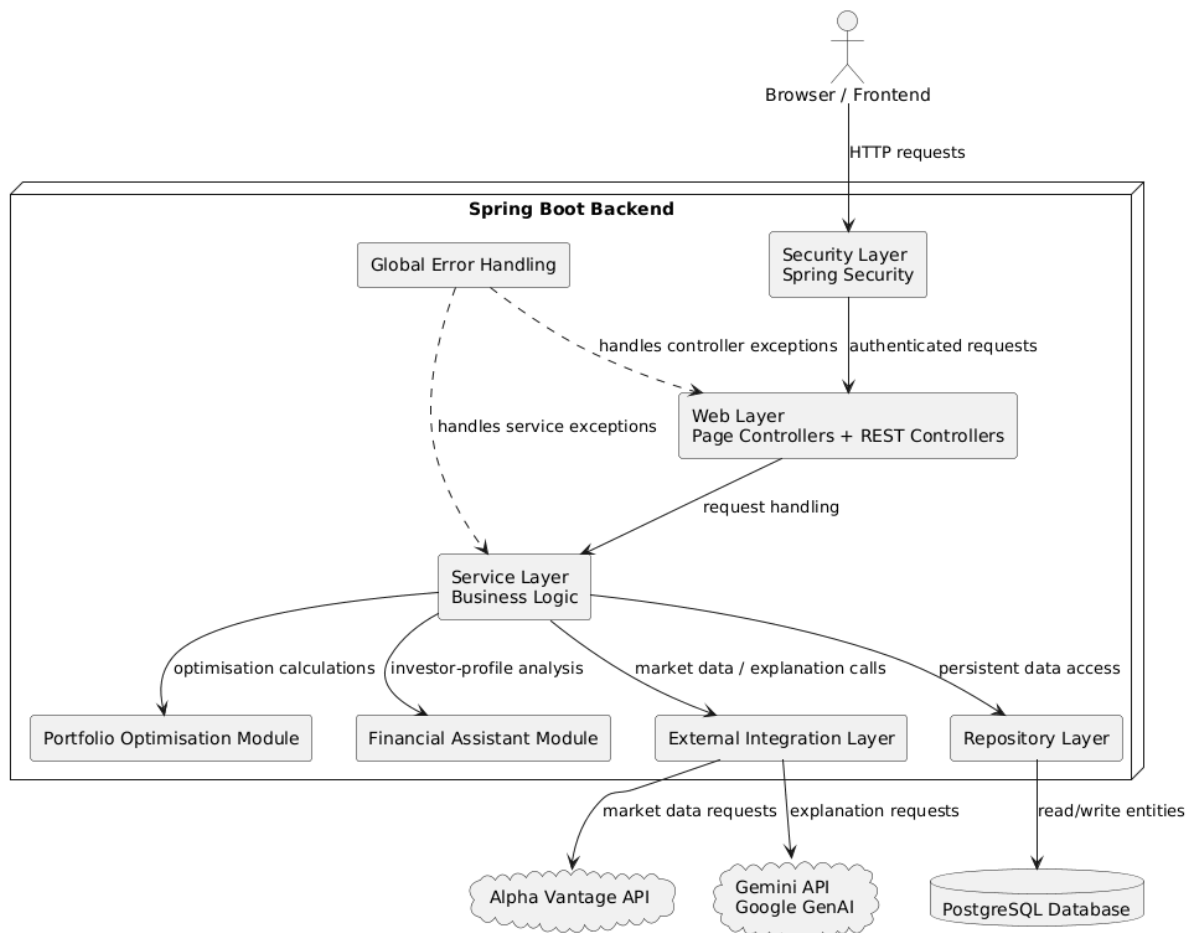


Figure 4.3: Backend Architecture Overview

4.5. Web and Security Layer Design

The web and security layer controls authenticated access to portfolio pages and API endpoints. Since portfolio data is user-specific, protected pages require authentication and services operate on the authenticated user context.

The web layer is divided into page controllers for Thymeleaf views and API controllers for AJAX/JSON requests. Controllers delegate business logic to services rather than performing calculations directly.

A global exception handler is included as a cross-cutting component. Its role is to convert known exceptions, invalid requests, missing data, and external service failures into structured API error responses. This prevents internal errors from being exposed directly to the frontend and supports more consistent error handling.

Figure 4.4 Web and Security Layer Design

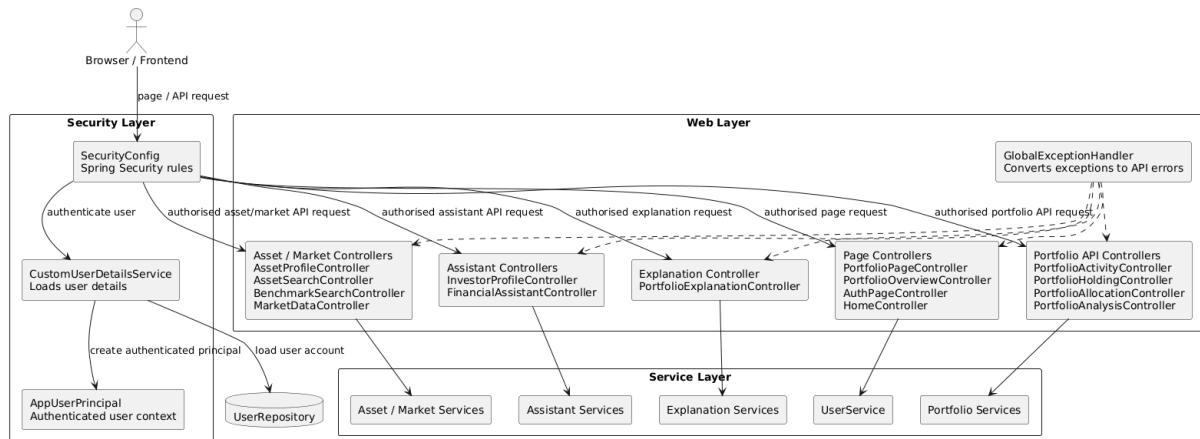


Figure 4.4: Web and Security Layer Design

4.6. Portfolio Services Design

The portfolio services implement the activity-driven portfolio model and provide data for overview, holdings, allocation, analysis, optimisation, explanation and PDF export features.

The activity service stores simulated BUY and SELL records, while the holdings service aggregates them into current quantities, average cost, market value and gain/loss where price data is available.

The allocation service groups derived holdings by holding, asset class, sector, market and currency.

The benchmark service compares portfolio performance against a selected benchmark over a chosen period.

The optimisation service obtains eligible holdings, retrieves historical prices and delegates calculation to the selected algorithm through a shared optimisation interface.

4.7. Financial Assistant Services Design

The financial assistant component was designed to extend the system beyond passive portfolio tracking. Its purpose is to use investor profile information to produce educational suitability insights. This component does not provide regulated financial advice. Instead, it compares the user's current portfolio allocation against a simplified target allocation derived from the user's profile.

The assistant workflow begins with the investor profile. Inputs such as horizon, goal, risk tolerance, experience and income stability are converted into a simplified risk profile and target allocation.

The assistant maps detailed asset classes into broader buckets, then compares actual allocation with the target allocation.

The suitability analysis component identifies major differences and provides assistant insights for the overview page.

Figure 4.6: Financial Assistant Services Design

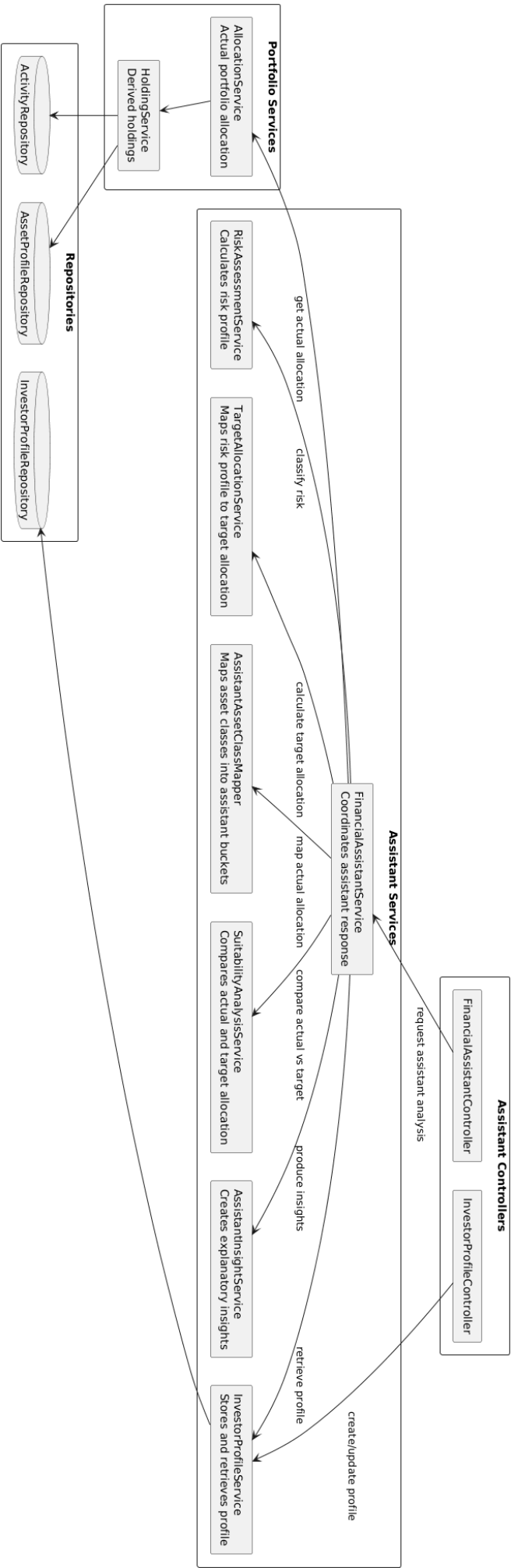


Figure 4.6 Financial Assistant Services Design

4.8 External Integration Design

The system depends on two external services: a market data provider and an LLM provider. To reduce coupling, both are accessed through integration layers rather than being called directly from controllers or unrelated services.

The market-data integration is designed around a provider abstraction so portfolio services request data through an internal service rather than depending directly on Alpha Vantage-specific code.

The explanation integration follows the same pattern: calculated portfolio context is sent through an LLM client abstraction to Gemini, keeping provider-specific details outside the core services.

Figure 4.7 External Integration Design

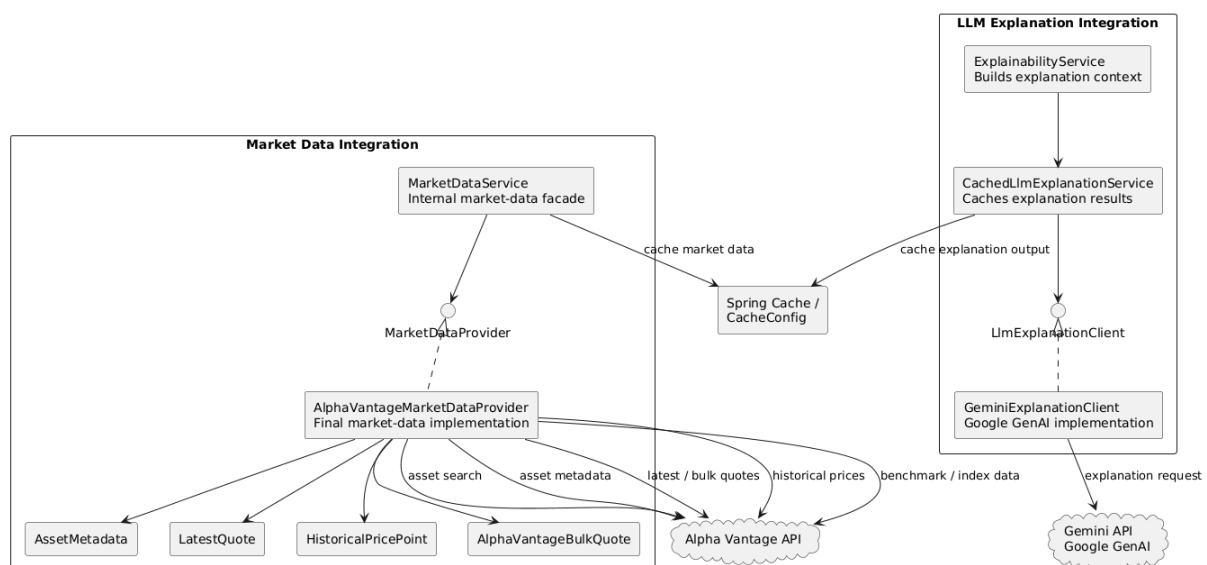


Figure 4.7: External Integration Design

4.9 Data Model, Persistence and Repository Design

The persistence design stores core user-owned data while deriving analytical outputs when needed. The main persisted entities are users, investment activities, asset profiles and investor profiles.

Investment activities are persisted because they are the source of truth. Holdings are recalculated from the activity history rather than stored as separate persistent records.

The repository layer provides the persistence boundary, while outputs such as holdings, allocation summaries, optimisation results and explanations are calculated when requested.

Figure 4.8 Persistence and Repository Design

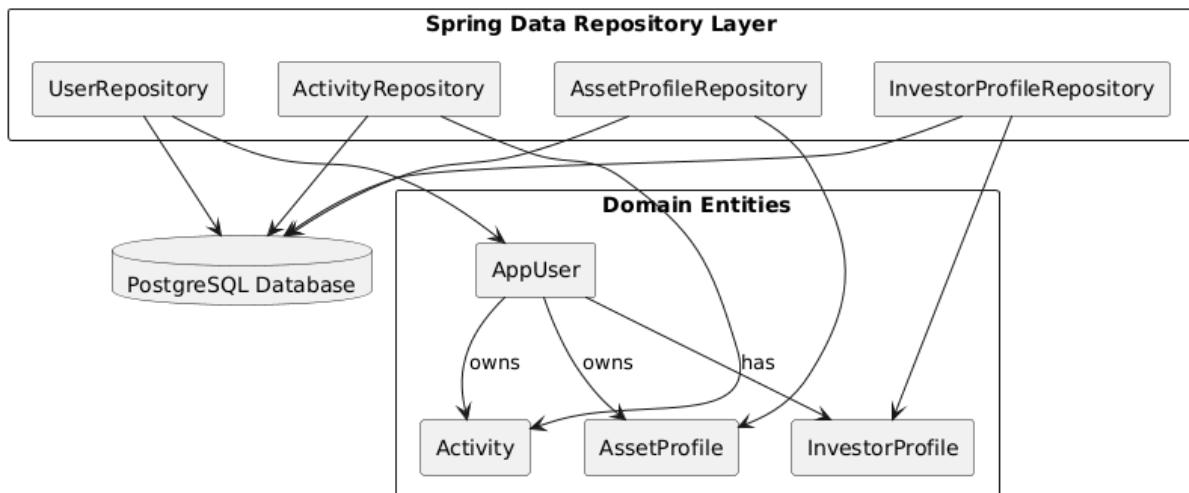


Figure 4.8: Data Model, Persistence and Repository Design

4.10 Activity-Driven Portfolio Workflow

The activity-driven workflow is one of the most important design decisions. Users record investment activities, and the system derives the current portfolio state from those records.

Derived holdings are used by allocation, benchmark comparison, optimisation, assistant insights, explanation and PDF export. This keeps downstream outputs consistent with the activity history.

The main flow is:

activities → holdings → allocation → analysis → selected optimisation algorithm → explanation/assistant insights/export

If the activity history changes, all downstream outputs can be recalculated from the same source of truth, keeping the whole workflow consistent.

Figure 4.9 Activity-Driven Portfolio Workflow

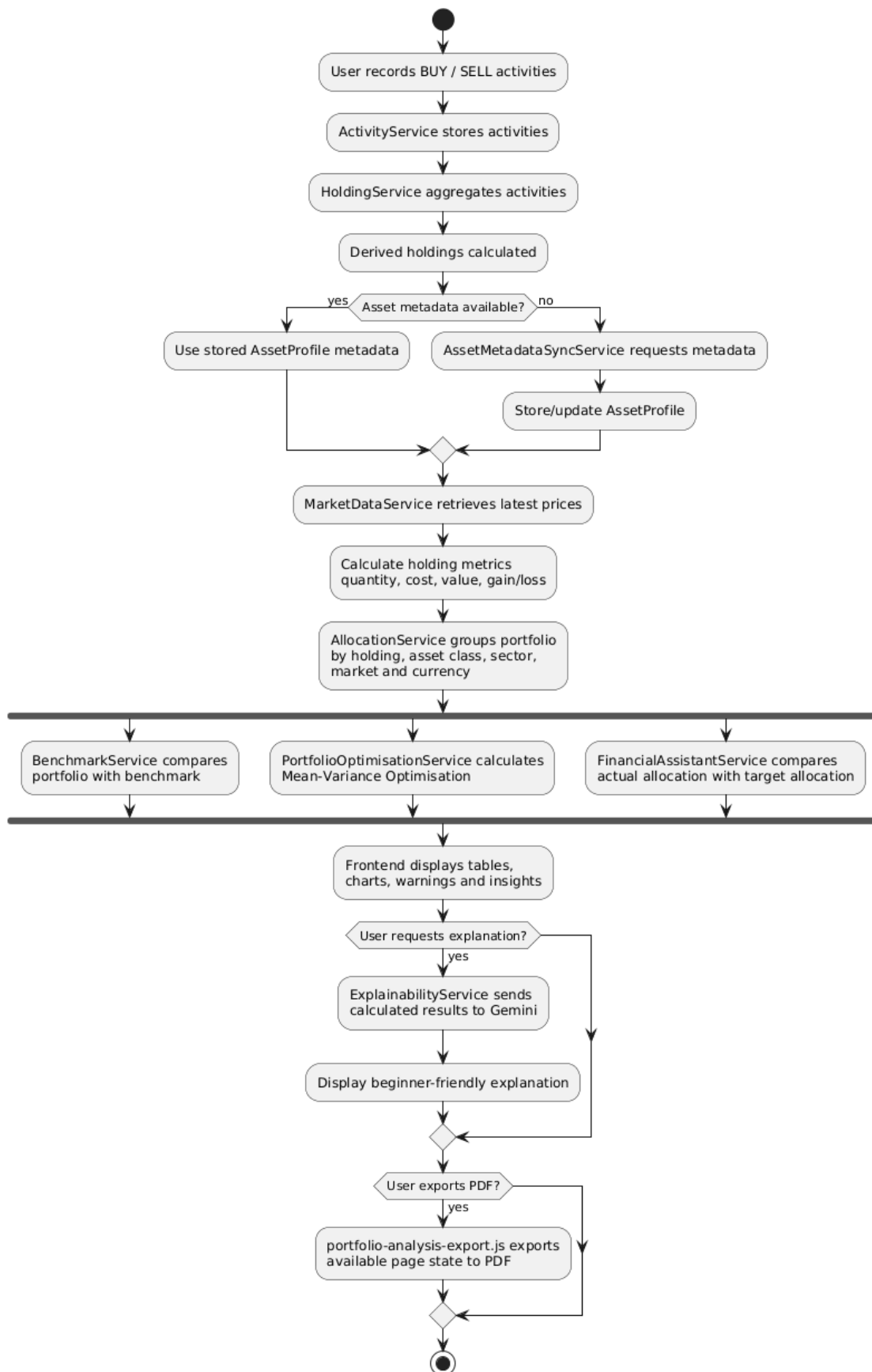


Figure 4.9: Activity-Driven Portfolio Workflow

4.11 Portfolio Optimisation Design

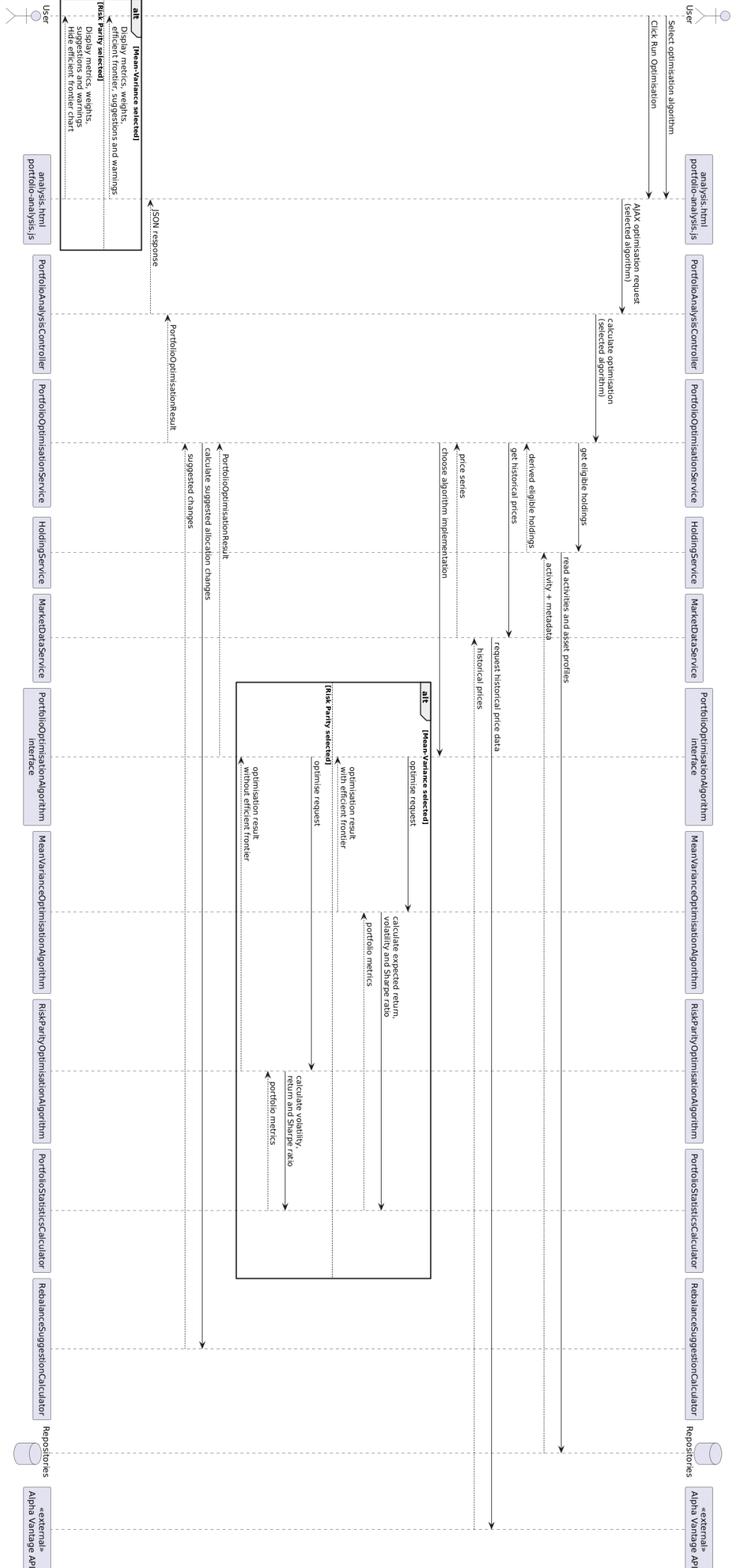
The portfolio optimisation design separates orchestration from algorithmic calculation. The optimisation service prepares inputs, while algorithm classes perform the selected calculation.

When the user requests optimisation, the backend retrieves derived holdings, filters eligible assets and retrieves historical prices. Assets with missing data or unsupported metadata are excluded with warnings.

For eligible assets, historical prices are converted into return series and passed to the selected algorithm. MVO returns risk-return metrics and efficient frontier points, while simplified Risk Parity returns inverse-volatility target weights.

After calculation, current and target weights are compared to produce educational suggested changes for the frontend.

Figure 4.10: Portfolio Optimisation Sequence Diagram



4.12 Explainability Design

The explanation design addresses the problem that portfolio metrics can be difficult for beginner users to interpret.

The system uses an LLM-based explanation layer, but the LLM does not calculate financial metrics. Backend services calculate the data first, and the explanation component converts the structured context into beginner-friendly text.

The explanation request is prepared by the backend and sent through the LLM integration layer; the returned text is displayed on the analysis page.

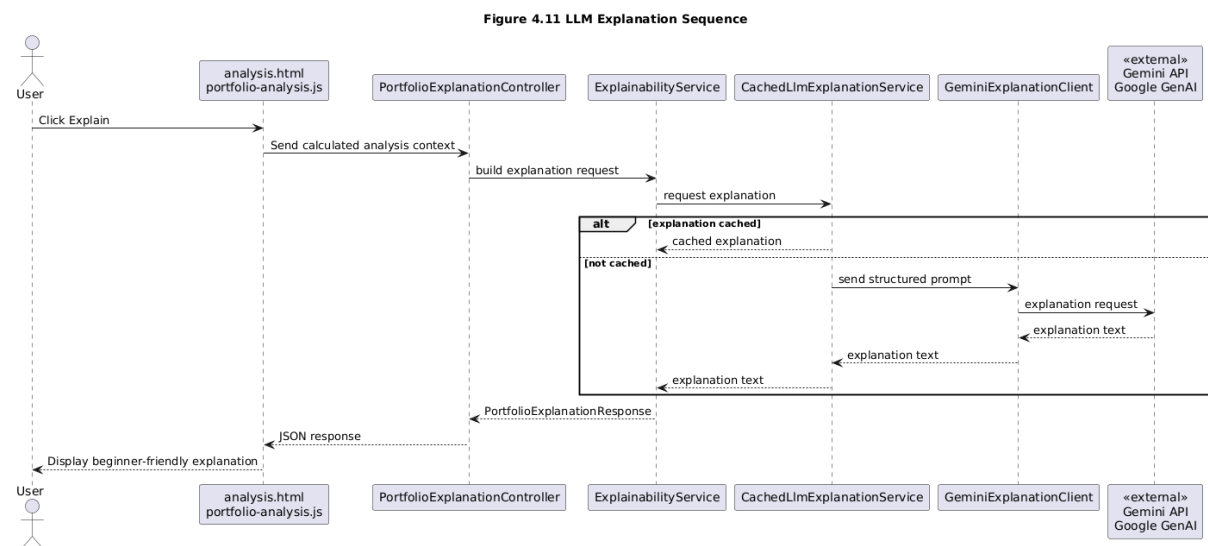


Figure 4.11: LLM Explanation Sequence Diagram

4.13 PDF Export Design

The PDF export feature was designed as a frontend-side reporting function that saves available analysis results for later review.

The export script reads the current analysis page state and includes available holdings, allocation, benchmark, optimisation, warning, chart and explanation sections.

The export logic respects the selected optimisation algorithm: MVO may include the efficient frontier chart, while Risk Parity omits it.

A key design constraint is that PDF export should not rerun expensive calculations or make new market-data/LLM requests; it exports results already available on the page.

4.14 Error Handling and Robustness

Error handling is important because the system depends on user input, market data, calculations and external services.

A global error-handling component converts known backend errors into structured API responses.

Robustness is also supported by validation and warnings, especially when assets cannot be included in optimisation.

LLM explanation is treated as a supporting feature, so portfolio calculation results remain available even if explanation fails.

4.15 Summary

This chapter described the architecture and design of the financial portfolio assistant. The system uses a layered Spring Boot architecture with a browser-based frontend, backend service layer, repository layer, external integration layer, portfolio optimisation module and Financial Consultant Assistant module. The main design decision is the activity-driven portfolio model, where user activities are treated as the source of truth and holdings are derived from those activities. Chapter 5 explains how these design decisions were implemented in the final system.

5. Implementation and Development Process

This chapter explains how the design described in Chapter 4 was implemented in the final system. The chapter is organised around the main implemented features rather than only separating the application into backend and frontend work. This is because the project's main contribution is the complete portfolio assistant workflow: authenticated users record investment activities, the backend derives holdings, portfolio services calculate allocation and analysis results, optimisation algorithms calculate alternative allocations, and the frontend presents the output through pages, charts, explanation text and PDF export.

The full source code is submitted through the project repository. Therefore, this chapter does not reproduce complete source files. Instead, it explains the implementation structure, identifies the main source-code file paths, and includes only short selected code snippets where they clarify an important algorithm or design decision. Detailed testing evidence is discussed separately in Chapter 6.

5.1 Implementation Overview

5.1.1 Code Organisation

The application was implemented as a Java Spring Boot web application. The codebase follows the layered architecture described in Chapter 4. Controllers handle page and API requests, services contain business logic, repositories provide database access, domain classes represent persistent entities and enums, DTOs define request and response objects, and integration classes isolate external providers such as Alpha Vantage and Gemini.

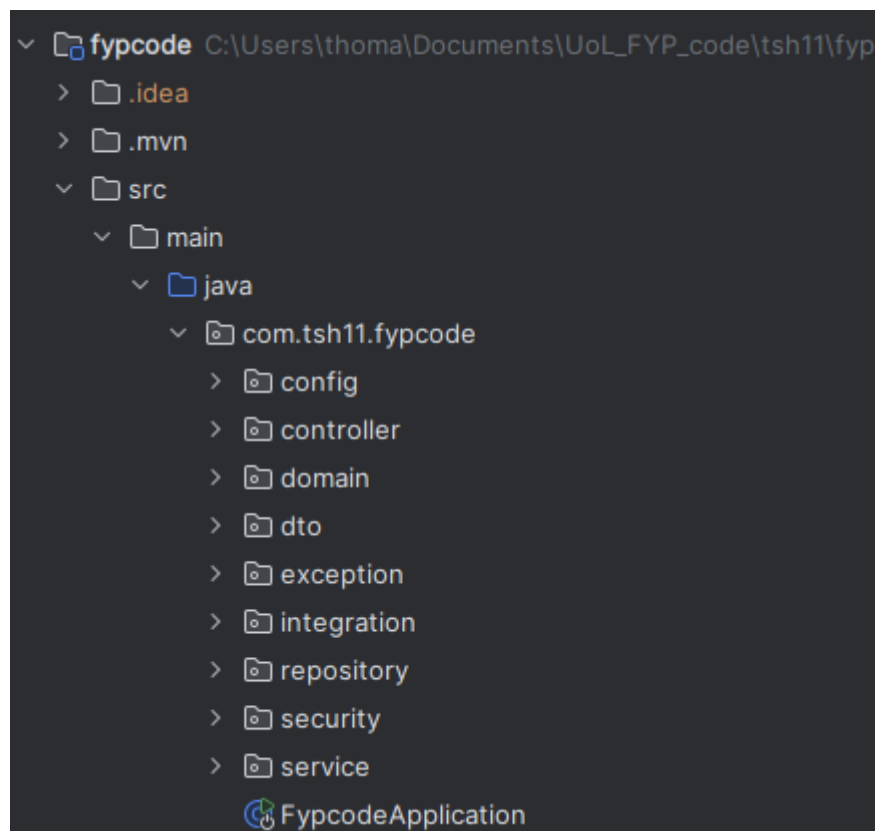


Figure 5.1: Backend file structure. Showing controllers, services, repositories, domain models, DTOs, security and integration packages.

The main implementation structure is summarised in Figure 5.1. The ``controller`` package contains page and REST/API controllers, while the ``service`` package contains the main business logic for portfolio calculation, market data, optimisation and explanation. More specialised services are separated into ``service/assistant`` for the Financial Consultant Assistant and ``service/portfolio`` for optimisation algorithms. Database access is handled through ``repository``, persistent entities and enums are stored under ``domain``, and external providers are isolated under ``integration``. Frontend templates are stored in ``src/main/resources/templates``, while page-specific JavaScript files are stored in ``src/main/resources/static/js``.

5.1.2 Development Process

Development followed an iterative feature-based process: core portfolio functions were implemented first, followed by market-data integration, analysis, optimisation, explanation, PDF export, assistant features and onboarding changes.

5.1.3 Use of Generative AI in Implementation

Generative AI was used in two parts during this project. First, it was used as a development aid for the initial implementation of the Mean-Variance Optimisation component. This component is therefore not treated as the main original coding contribution of the project. In this chapter it is discussed mainly in terms of how it was integrated into the wider portfolio optimisation workflow, exposed through the shared optimisation interface, and connected to the analysis page.

Second, generative AI is used as a runtime feature of the final system through the Gemini/Google GenAI explanation integration. In this case, the LLM is not used to calculate portfolio metrics or make independent financial decisions. The backend first calculates portfolio values, allocation, benchmark results, optimisation outputs, warnings and suitability information. These calculated results are then passed to the LLM so that it can produce beginner-friendly explanatory text.

Because the Mean-Variance Optimisation component was developed with generative AI assistance, its correctness could not be assumed. Its behaviour was reviewed against the finance concepts discussed in Chapter 3 and evaluated using automated tests and manual analysis-page checks. The detailed testing and evaluation of this component are discussed in Chapter 6.

5.1.4 Key Implementation Challenges

The main implementation challenges were user-specific data separation, activity-derived holdings, rate-limited market data, incomplete metadata, optimisation input validity and beginner-friendly presentation.

5.2 Backend Implementation

5.2.1 Authentication and User-Specific Data

Authentication was implemented using Spring Security, with an application-specific principal so controllers and services can identify the current user.

The main implementation is located in the security configuration, custom user details service, authenticated principal, user service and user repository.

5.2.2 Investment Activity Management

Investment activities are the starting point of the workflow. Users can create, view, update and delete simulated BUY and SELL activities, which are persisted as the source of truth.

The main implementation is located in the related domain, controller, service, repository, template and JavaScript files for this feature.

5.2.3 Holdings Calculation from Activities

Holdings are derived rather than stored. The holdings calculation service groups the authenticated user's activities by asset symbol and aggregates BUY and SELL transactions to calculate the current quantity, total cost and average cost. Where market price data is available, the service also calculates latest value and unrealised gain or loss.

The main implementation is located in the holdings service, holdings controller, response DTO, holdings template and page JavaScript.

The key implementation decision is that activities remain the persistent source of truth. Code Snippet 5.1 shows the central part of this implementation: activities are retrieved for the current user, grouped by normalised symbol, and BUY and SELL records update quantity and cost basis differently.

Code Snippet 5.1: Deriving holdings from investment activities.

Source: `src/main/java/com/tsh11/fypcode/service/HoldingService.java`

```
@Cacheable(value = "holdingsByUser", key = "#userId", sync = true)
public List<HoldingResponse> getHoldings(UUID userId) {
    List<Activity> activities = activityRepository.findByUserIdOrderByDateDesc(userId);

    Map<String, List<Activity>> grouped = activities.stream()
        .filter(activity -> activity.getSymbol() != null && !activity.getSymbol().isBlank())
        .filter(activity -> activity.getType() == ActivityType.BUY || activity.getType() ==
ActivityType.SELL)
        .collect(Collectors.groupingBy(activity -> normalizeSymbol(activity.getSymbol())));

    for (Activity activity : symbolActivities) {
        if (activity.getType() == ActivityType.BUY) {
            BigDecimal buyCost = buyQuantity.multiply(unitPrice).add(fee);
            quantity = quantity.add(buyQuantity);
            costBasis = costBasis.add(buyCost);
        } else if (activity.getType() == ActivityType.SELL) {
            BigDecimal avgCost = costBasis.divide(quantity, SCALE, RoundingMode.HALF_UP);
            BigDecimal removedCost = avgCost.multiply(effectiveSellQuantity);
            quantity = quantity.subtract(effectiveSellQuantity);
            costBasis = costBasis.subtract(removedCost);
        }
    }
}
```

This keeps holdings consistent with the activity history.

5.2.4 Asset Profiles and Metadata Synchronisation

Asset profiles store metadata such as symbol, display name, asset class, sector, market, currency and support flags.

The main implementation is located in the related domain, controller, service, repository, template and JavaScript files for this feature.

5.2.5 Allocation Analysis

The allocation feature calculates how the current portfolio is distributed across holdings, asset classes, sectors, markets and currencies. It uses the derived holdings from the holdings service and groups their current values into allocation categories. The backend returns structured allocation responses, while the frontend renders charts and tables.

The main implementation is located in the related domain, controller, service, repository, template and JavaScript files for this feature.

This supports the educational aim by showing concentration by category.

5.2.6 Benchmark Comparison

Benchmark comparison is implemented as part of the analysis workflow. It compares portfolio performance against a selected benchmark over a date range and returns metrics and chart points.

The main implementation is located in the related domain, controller, service, repository, template and JavaScript files for this feature.

``src/main/java/com/tsh11/fypcode/controller/PortfolioAnalysisController.java``

The benchmark feature is a supporting analysis tool because a single benchmark may not fully represent a mixed-asset portfolio.

5.2.7 Portfolio Optimisation

Portfolio optimisation is implemented through a shared optimisation workflow. The analysis controller receives an optimisation request from the frontend. The optimisation service prepares common inputs such as derived holdings, eligible assets, current weights, historical return series and risk-free rate. The calculation is then delegated to the selected optimisation algorithm.

5.2.7.1 Shared Optimisation Interface

The optimisation module is designed around a common algorithm interface. This allows the optimisation service to call MVO and simplified Risk Parity through the same contract.

This design was important because it made it possible to add simplified Risk Parity after Mean-Variance Optimisation without redesigning the controller or the analysis page workflow.

5.2.7.2 Mean-Variance Optimisation Integration

The Mean-Variance Optimisation component implements the shared optimisation interface. It receives prepared request data from the optimisation service and returns portfolio metrics, target weights, efficient frontier points, suggested changes and warnings. The main source file is ``src/main/java/com/tsh11/fypcode/service/portfolio/MeanVarianceOptimisationAlgorithm.java``.

Because this component was developed with generative AI assistance, Chapter 6 focuses on its verification.

5.2.7.3 Risk Parity / Inverse-Volatility Weighting

A second algorithm, simplified Risk Parity, was implemented manually to demonstrate that the optimisation module can support more than one portfolio construction approach. The source file is ``src/main/java/com/tsh11/fypcode/service/portfolio/RiskParityOptimisationAlgorithm.java``.

This implementation uses inverse-volatility weighting. It calculates annualised volatility for each eligible asset from historical returns, converts volatility into an inverse-volatility score, normalises the scores so the target weights sum to one, and then returns a target allocation. Code Snippet 5.3 shows the core logic.

Code Snippet 5.3: Simplified Risk Parity inverse-volatility weighting.

Source: `src/main/java/com/tsh11/fypcode/service/portfolio/RiskParityOptimisationAlgorithm.java`

```
private Map<String, BigDecimal> calculateInverseVolatilityWeights(
    List<String> symbols,
    Map<String, List<BigDecimal>> dailyReturns) {
```

```

Map<String, BigDecimal> inverseVolatilities = new LinkedHashMap<>();
BigDecimal totalInverseVolatility = ZERO;

for (String symbol : symbols) {
    BigDecimal annualisedVolatility = calculateAnnualisedVolatility(dailyReturns.get(symbol));
    if (annualisedVolatility.compareTo(ZERO) <= 0) {
        throw new IllegalArgumentException("Risk parity cannot be calculated because "
            + symbol + " has zero or invalid volatility.");
    }

    BigDecimal inverseVolatility = ONE.divide(annualisedVolatility, MC);
    inverseVolatilities.put(symbol, inverseVolatility);
    totalInverseVolatility = totalInverseVolatility.add(inverseVolatility, MC);
}

Map<String, BigDecimal> weights = new LinkedHashMap<>();
for (String symbol : symbols) {
    weights.put(symbol, inverseVolatilities.get(symbol).divide(totalInverseVolatility, MC));
}
return weights;
}

```

Higher-volatility assets therefore receive lower target weights, while invalid inputs such as missing returns or zero volatility are rejected.

5.2.7.4 Algorithm Selection and Output Handling

The frontend analysis page includes an algorithm selection control. When the user runs optimisation, the selected algorithm is sent to the backend. The backend uses this algorithm type to select the appropriate implementation. The output handling differs between the two algorithms. Mean-Variance Optimisation can return efficient frontier points, while simplified Risk Parity returns one inverse-volatility target allocation and does not calculate a full efficient frontier.

This difference is also handled in the frontend: the efficient frontier is displayed for MVO but hidden for Risk Parity, and PDF export follows the same rule.

5.2.8 Financial Consultant Assistant

The Financial Consultant Assistant collects investor profile data, calculates a risk profile and target allocation, compares actual allocation against the target and produces assistant insights.

The main implementation is located in the related domain, controller, service, repository, template and JavaScript files for this feature.

The assistant is rule-based, educational and not regulated financial advice.

Financial Consultant Assistant

Rule-based suitability insights based on your investor profile and current allocation.

[Update Profile](#)

Risk Profile

Balanced

Risk score: 47/100

Target Allocation

Equity: 60.00%

Bond: 30.00%

Cash: 10.00%

Actual Allocation

Equity: 100.00%

Bond: 0.00%

Cash: 0.00%

Crypto: 0.00%

Real estate: 0.00%

Other: 0.00%

Unclassified: 0.00%

Bucket	Target	Actual	Difference	Status
Equity	60.00%	100.00%	+40.00%	Significantly Overweight
Bond	30.00%	0.00%	-30.00%	Significantly Underweight
Cash	10.00%	0.00%	-10.00%	Slightly Underweight

Figure 5.2: Example Financial Consultant Assistant output showing profile-based portfolio insight.

5.2.9 Natural-Language Explanation

Natural-language explanation is implemented as a backend service that sends calculated analysis context to Gemini through the LLM integration layer.

The main implementation is located in the related domain, controller, service, repository, template and JavaScript files for this feature.

This follows the design boundary in Chapter 4: calculation is deterministic and explanation is separate.

5.2.10 Caching and API Quota Handling

Market-data calls are a practical constraint because external APIs can be rate-limited, so the implementation includes provider abstraction, caching and request reduction.

The main implementation is located in the related domain, controller, service, repository, template and JavaScript files for this feature.

5.2.11 Error Handling and Warnings

The system includes centralised error handling so known failures are returned as structured responses rather than raw exceptions.

The main implementation is located in the global exception handler, API error DTO and custom exception classes.

Warnings are used when assets cannot be included in optimisation due to missing historical data, unsupported metadata, zero value or unreliable pricing.

5.3 Frontend Implementation

5.3.1 Frontend Code Organisation

The frontend uses Thymeleaf templates, Bootstrap, page-specific JavaScript, AJAX requests and Chart.js visualisations.

Important frontend files include the shared portfolio header, portfolio templates and page-specific JavaScript under ``src/main/resources/templates`` and ``src/main/resources/static/js``.

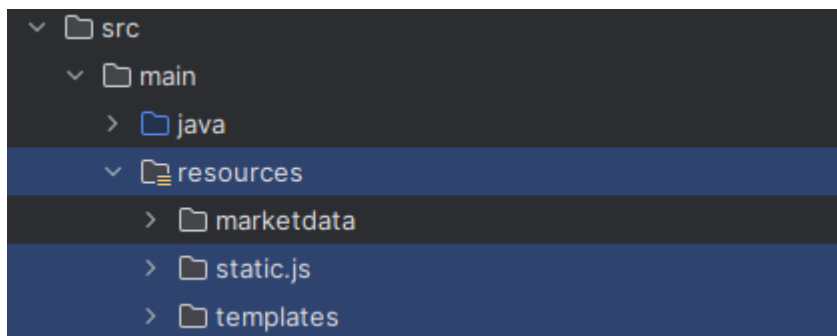


Figure 5.3: Frontend source-code organisation showing Thymeleaf templates, shared fragments and page-specific JavaScript files.

5.3.2 Shared Portfolio Header and Navigation

A shared portfolio header fragment provides consistent navigation, page title, user greeting and logout behaviour across portfolio pages.

The shared header fragment is used by the overview, investor profile, activities, asset profiles, holdings, allocation and analysis pages. The visible user greeting clarifies which account is active, and the logout button provides an obvious way to end the session.



Figure 5.4: Shared portfolio header showing the logged-in user, logout button and navigation links.

5.3.3 New-User Onboarding Flow

The frontend includes a UI-only setup guide for new users. Before profile and activity setup is complete, analysis-focused pages are visually disabled as guidance, not as security enforcement.

This is intentionally a usability guide rather than a security restriction. Direct URL access is not blocked. The purpose is to guide beginner users toward the intended workflow without adding risky redirects or backend restrictions close to submission.

A model advice component provides template-level setup state, including whether the user has an investor profile, whether the user has activities and whether setup is complete.

The overview page also includes an onboarding popup for users with no investor profile and no activities. The popup encourages the user to complete the investor

profile first and can be dismissed for the current browser session.

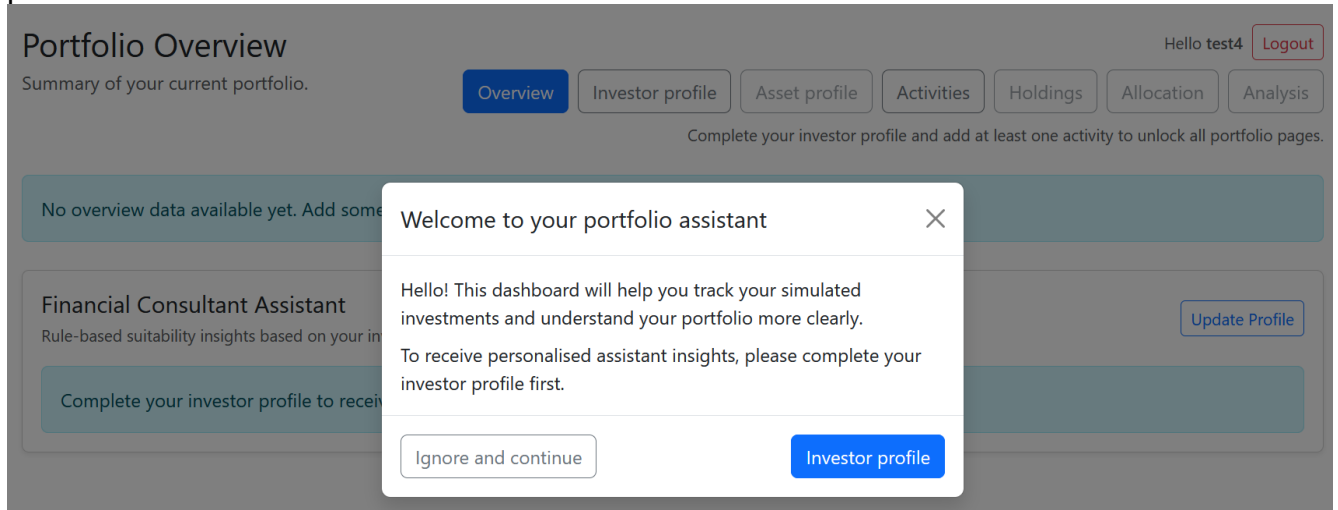


Figure 5.5: New-user onboarding popup shown when no investor profile and no activities exist.

5.3.4 Investor Profile Page and Help Popovers

The investor profile page collects information required by the assistant layer and includes help popovers for terms such as risk tolerance, experience and investment goal.

The main frontend files are ``src/main/resources/templates/portfolio/investor-profile.html`` and ``src/main/resources/static/js/portfolio-investor-profile.js``.

Investor Profile

Create a risk profile for educational portfolio suitability guidance.

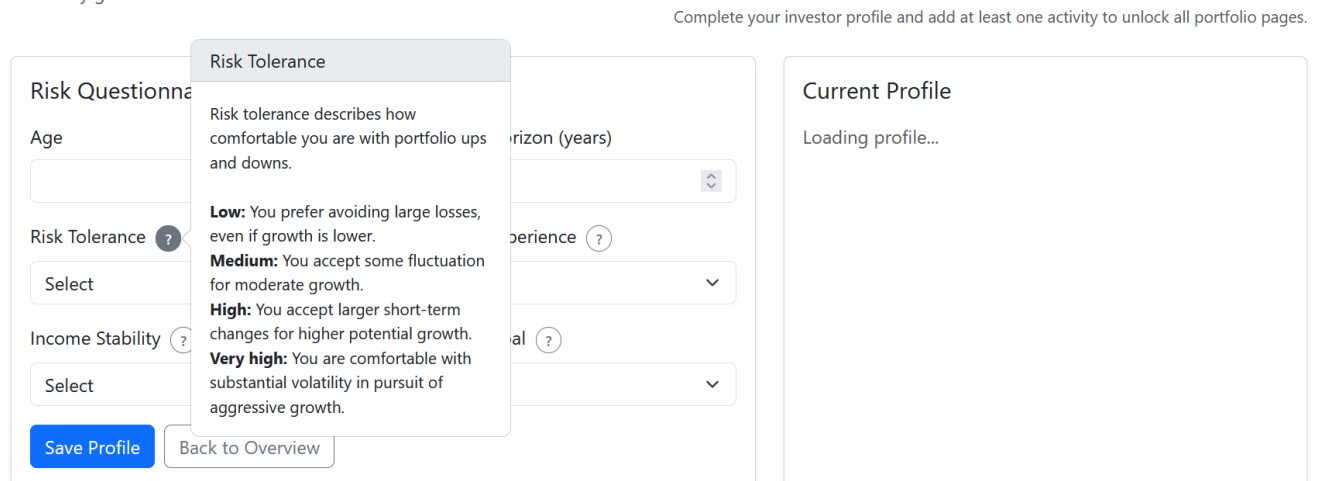


Figure 5.6: Investor profile page with a help popover explaining a questionnaire field.

5.3.5 Activities, Holdings and Allocation Pages

The activities, holdings and allocation pages implement the main tracking workflow: users enter transactions, holdings are derived, and allocation charts show portfolio distribution.

The frontend files for these pages are ``portfolio-activities.js``, ``portfolio-holdings.js`` and ``portfolio-allocation.js``. The allocation page uses Chart.js to present the calculated

allocation data visually. These pages demonstrate the activity-driven design: users input activities, and the rest of the portfolio view is calculated from them.

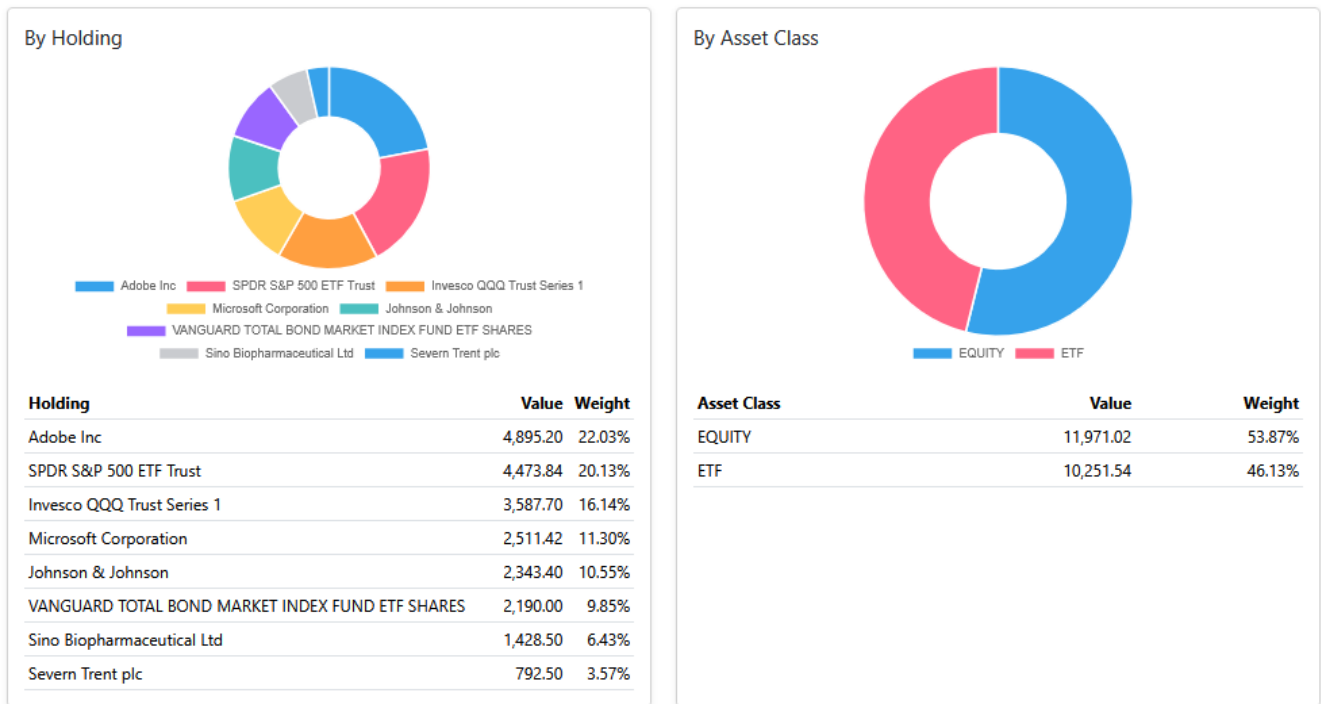


Figure 5.7: Allocation page visualising derived holdings by portfolio category.

5.3.6 Analysis Page: Benchmark, Optimisation and Explanation

The analysis page combines benchmark comparison, selectable optimisation algorithms, warnings, charts, suggested allocation changes and natural-language explanation.

The page includes an algorithm dropdown with Mean-Variance Optimisation and Risk Parity. When Mean-Variance Optimisation is selected, the efficient frontier chart can be shown. When Risk Parity is selected, the efficient frontier card is hidden because Risk Parity returns one inverse-volatility allocation rather than a full frontier. This frontend behaviour reflects the algorithmic difference described in Chapter 3 and Chapter 4.

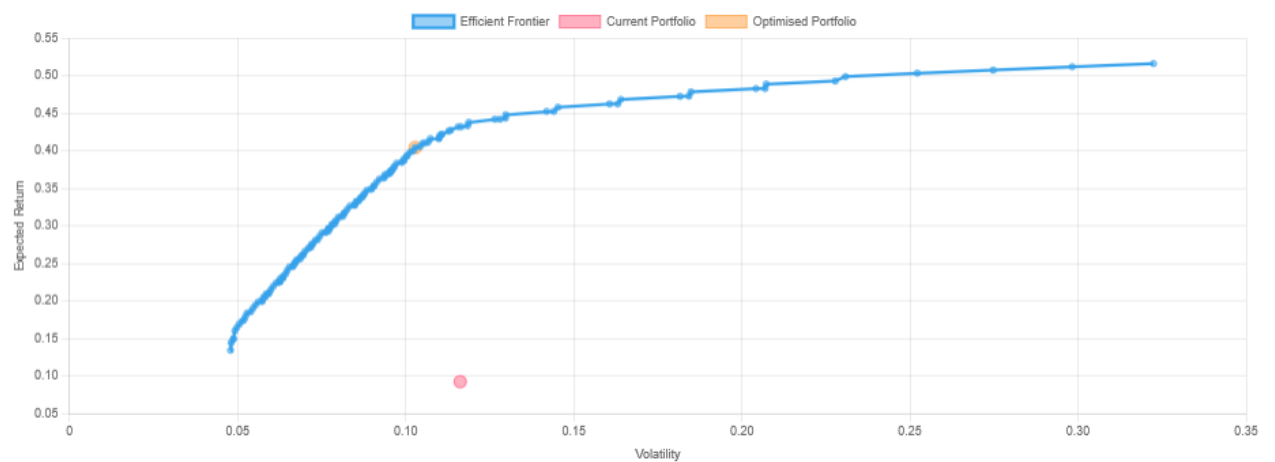
Portfolio Optimisation

Analyse the current portfolio using the selected optimisation algorithm.

From	To	Algorithm	Run Optimisation
24 / 05 / 2025	24 / 05 / 2026	Mean Variance	
Current Return 9.25%	Current Volatility 11.63%	Current Sharpe Ratio 0.6236	
Optimised Return 40.43%	Optimised Volatility 10.28%	Optimised Sharpe Ratio 3.7364	
		Sharpe ratio improved by 3.1128	

Efficient Frontier

The efficient frontier shows alternative mean-variance portfolios with different risk-return trade-offs.



Allocation Comparison

Asset	Current Weight	Target Weight	Difference	Suggested Change
ADBE	22.03%	0.00%	-22.03%	- 20.00 units (4,895.20)
BND	9.85%	0.00%	-9.85%	- 30.00 units (2,190.00)
JNJ	10.55%	50.00%	+39.45%	+ 37.42 units (8,767.88)
MSFT	11.30%	0.00%	-11.30%	- 6.00 units (2,511.42)
QQQ	16.14%	35.00%	+18.86%	+ 5.84 units (4,190.20)
SBHMY	6.43%	5.00%	-1.43%	- 22.22 units (317.37)
SPY	20.13%	0.00%	-20.13%	- 6.00 units (4,473.84)
SVTRF	3.57%	10.00%	+6.43%	+ 36.08 units (1,429.76)

Figure 5.8: Mean-Variance Optimisation result with efficient frontier output visible.

Portfolio Optimisation

Analyse the current portfolio using the selected optimisation algorithm.

From	To	Algorithm	Run Optimisation
24/05/2025	24/05/2026	Risk Parity	
Current Return 9.25%	Current Volatility 11.63%	Current Sharpe Ratio 0.6236	
Optimised Return 14.29%	Optimised Volatility 6.28%	Optimised Sharpe Ratio 1.9563 Sharpe ratio improved by 1.3326	

Allocation Comparison

Asset	Current Weight	Target Weight	Difference	Suggested Change
ADBE	22.03%	5.13%	-16.90%	- 15.34 units (3,754.89)
BND	9.85%	43.55%	+33.69%	+ 102.56 units (7,487.22)
JNJ	10.55%	10.01%	-0.53%	- 0.50 units (118.13)
MSFT	11.30%	6.97%	-4.34%	- 2.30 units (963.36)
QQQ	16.14%	10.36%	-5.78%	- 1.79 units (1,284.49)
SBHMY	6.43%	3.13%	-3.30%	- 51.34 units (733.42)
SPY	20.13%	13.93%	-6.20%	- 1.85 units (1,378.53)
SVTRF	3.57%	6.92%	+3.36%	+ 18.82 units (745.59)

Figure 5.9: Risk Parity optimisation result with the efficient frontier hidden.

Explanation

[Explain This Result](#)[What is a finance portfolio?](#)[How is this portfolio calculated?](#)

Portfolio Explanation

The calculation suggests a new way to arrange your investments that could give you more money back with fewer big swings.

Right now, your investments have a "current portfolio" setup. This setup has an expected yearly return [possible yearly gain estimated by the calculation] of 9.2%. This means the calculation thinks you might gain about 9.2% on your money each year. The "volatility" [how much the value may move up and down] is 11.6%. This shows how much your investment value might bounce around. Your "Sharpe ratio" [more return for the amount of ups and downs] is 0.62. A higher number here is generally better, as it means you are getting more gain for the amount of ups and downs you might experience.

The system ran calculations to find a different way to set up your investments. This is called the "calculated portfolio". This new setup has a much higher expected yearly return of 14.3%. So, the calculation suggests you could gain 5% more each year. The volatility of this calculated portfolio is much lower at 6.3%. This means the value of your investments might move up and down less, by 5.3% less. The Sharpe ratio for the calculated portfolio is also much higher at 1.96. This means the system suggests a way to get a lot more return for the amount of ups and downs.

To reach this calculated portfolio, the system suggests making some changes to where your money is placed. This is about changing the "asset weight" [how much of your total money is in each specific investment]. For example, the calculation suggests you significantly increase your money in BND. It suggests moving from 9.9% to 43.5% of your total investments, which is a big jump of 33.7%. On the other hand, it suggests reducing your money in ADBE quite a bit. It moves from 22% down to 5.1%, a drop of 16.9%. It also suggests reducing your money in MSFT, QQQ, SBHMY, and SPY by smaller amounts. The system suggests slightly increasing your money in SVTRF, moving from 3.6% to 6.9%. It keeps JNJ almost the same.

The calculation did not provide any specific warnings for this suggested portfolio.
This is an explanation of the calculation, not financial advice.

Figure 5.10: Natural-language explanation generated from backend-calculated analysis results.

5.3.7 PDF Export

PDF export is implemented as a frontend-side reporting function that uses jsPDF and jspdf-autotable to save the current analysis page state.

The export feature is designed not to rerun expensive calculations automatically. If benchmark comparison, optimisation or explanation has not been calculated in the current page session, the PDF indicates that the section is unavailable rather than silently making new API calls. This prevents unexpected market-data or LLM requests during export.

The export logic also respects the selected optimisation algorithm. If Mean-Variance Optimisation was used, the efficient frontier chart can be included. If Risk Parity was used, the efficient frontier chart is omitted because that algorithm does not calculate a full efficient frontier.

5.4 Summary

This chapter described how the system was implemented. The backend uses a layered Spring Boot structure with controllers, services, repositories, domain models, DTOs and integration components. The portfolio workflow is activity-driven: activities are persisted, holdings are derived, and downstream features such as allocation, benchmark comparison, optimisation, explanation and PDF export use the calculated portfolio state. The testing and evaluation of this implementation are discussed in Chapter 6.

6. Testing and Evaluation

This chapter evaluates the implemented financial portfolio assistant against the requirements defined in Chapter 2 and the design and implementation described in Chapters 4 and 5. The evaluation combines automated tests, coverage inspection, integration and regression checks, and one automated browser-level end-to-end test. The purpose of this chapter is not only to show that tests were written, but also to explain what risks were tested and why those tests are relevant to the final system.

The test evidence focuses on the main project risks: whether activities are correctly converted into holdings, whether allocation and assistant logic produce expected outputs, whether portfolio optimisation works for both Mean-Variance Optimisation and simplified Risk Parity, whether API endpoints return suitable responses, whether known market-data and quota problems were addressed, and whether the main user-facing workflow works in a browser. No test code snippets are reproduced in this chapter because the full test source code is submitted in the repository. Instead, the chapter refers to test file paths, tables, coverage results, and appendices.

6.1 Testing Strategy

The testing strategy was layered. Unit tests were used for deterministic calculation and rule-based logic, controller and API tests were used for request/response behaviour, integration tests were used for interactions between components, regression tests were used to check previously discovered failure cases, and a Playwright test was used to validate the main user workflow through a browser [26]. This combination was necessary because the project contains different types of behaviour: numerical calculation, database-backed user data, external API integration, dynamic web pages, PDF export, and LLM-generated explanation.

Automated tests were prioritised for deterministic logic, while visual and qualitative outputs such as PDF layout and LLM explanation readability were checked through inspection and supporting evidence.

Table 6.1: Testing strategy overview.

Test level	Purpose	Example areas tested
Unit testing	Verify deterministic service logic and algorithmic behaviour.	Holdings calculation, allocation analysis, risk assessment, target allocation, suitability comparison, optimisation algorithms.
Controller/API testing	Verify endpoint status codes, validation behaviour and JSON response structures.	Holdings, allocation, analysis, asset search, investor profile and assistant endpoints.
Integration testing	Verify that multiple components work together.	Activity-to-holdings workflow, market-data failure handling, optimisation eligibility.
Regression testing	Ensure that previously identified bugs and risks do not reappear.	API quota handling, benchmark comparison, optimisation eligibility, cache-related behaviour.
Automated end-to-end testing	Verify a representative user workflow through a real browser.	Login, holdings, allocation, analysis, Mean-Variance Optimisation, Risk Parity, PDF export.
Coverage inspection	Quantify how much of the codebase was exercised by automated tests.	Overall coverage and package-level coverage for core calculation packages.

Table 6.2: Black-box and white-box testing methods used in the project.

Testing method	Applied to	Purpose in this project
Equivalence partitioning	Holdings, assistant services, benchmark, optimisation and Risk Parity tests.	Check representative valid and invalid input groups without needing every possible input.
Boundary value analysis	Holdings, allocation, risk assessment, benchmark and optimisation tests.	Check edge cases such as zero quantities, empty lists, missing data and minimum data requirements.
Category partitioning	Allocation and asset metadata tests.	Check grouping by categories such as asset class, sector, market, currency and support type.
Branch testing	Holdings, assistant services, metadata sync, portfolio optimisation and Risk Parity tests.	Check important decision paths such as BUY/SELL, supported/unsupported assets, valid/invalid input, and warning paths.
Loop testing	Holdings, allocation, portfolio optimisation, portfolio statistics, MVO and Risk Parity tests.	Check no-item, one-item and multiple-item scenarios.
Regression testing	Benchmark, holding eligibility, market data quota and optimisation quota tests.	Check that previously discovered issues remain fixed.

6.2 Test Code Organisation

The test code is organised under ``src/test/java/com/tsh11/fypcode/'`. The structure separates controller tests, service tests, integration tests, regression tests and browser-level end-to-end tests. Calculation-heavy services are further divided into black-box and white-box test packages, so that input-output behaviour and internal control-flow behaviour can both be evaluated.

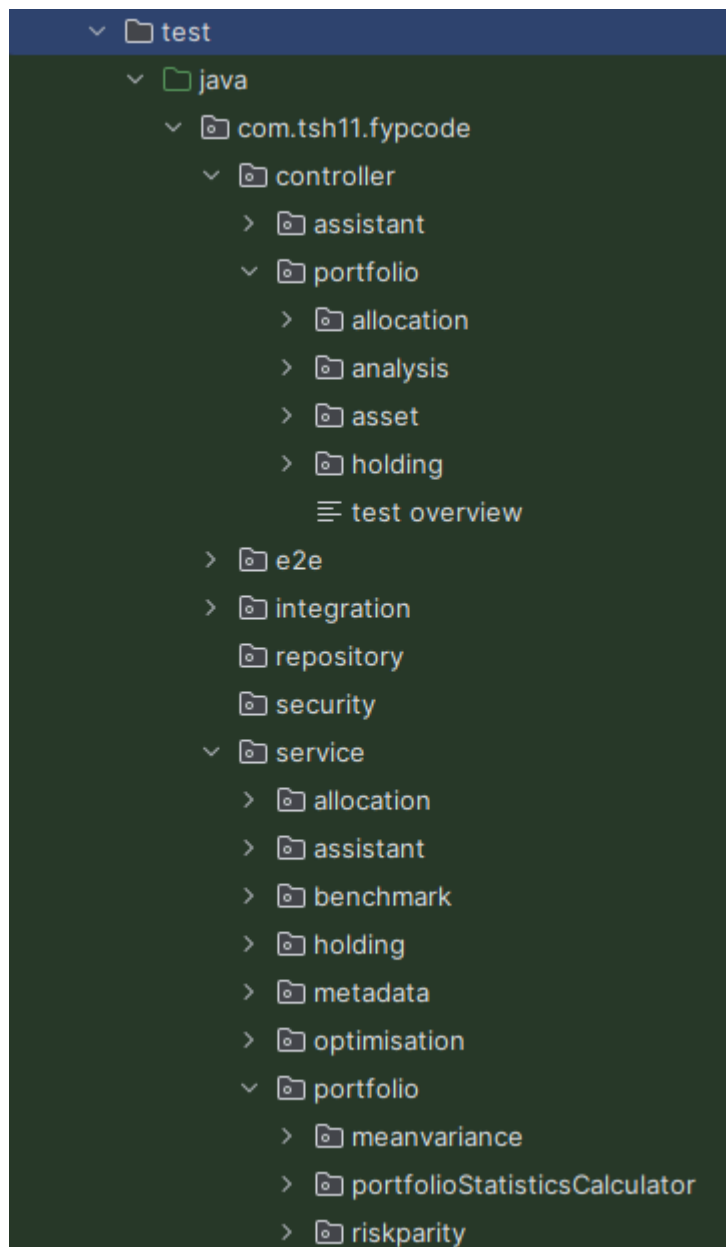


Figure 6.1: Test source-code organisation showing controller, service, integration, regression and end-to-end test areas.

6.3 Requirements Evaluation Plan

The requirements evaluation plan links Chapter 2 requirements to expected behaviour and testing methods. The tables below reflect the final implementation, including Risk Parity, Playwright E2E testing and PDF export evidence.

6.3.1 Essential Requirements

Table 6.4: Evaluation plan for essential requirements.

Requirement	Expected behaviour	Testing method
User authentication	Users register, log in and access only their own portfolio data.	Controller/security tests and E2E login step.
Investment management activity	Users create, view, update and delete simulated activities.	Controller/API and service tests.

Derive holdings from activities	BUY/SELL activities produce derived holdings.	Holding service black-box, white-box and regression tests.
Calculate holding metrics	Quantity, cost, average cost, value and gain/loss are calculated where data is available.	Holding service tests with known inputs.
Market data integration	Supported search, metadata, latest, historical and benchmark data are retrieved or handled safely.	Integration, failure and quota regression tests.
Asset allocation visualisation	Allocation by holding, asset class, sector, market and currency is displayed.	Allocation tests and frontend evidence.
Benchmark comparison	Portfolio and benchmark returns are compared over a selected period.	Benchmark service and regression tests.
Portfolio optimisation using selectable algorithms	MVO returns metrics/frontier; Risk Parity returns inverse-volatility weights without frontier.	MVO, Risk Parity, optimisation service and E2E tests.
User interface	Main portfolio pages are available and usable.	Screenshots, controller tests and Playwright E2E.
Error handling and warnings	Invalid input, missing data and external failures produce warnings/errors.	Controller, exception, failure and regression tests.

6.3.2 Recommended Requirements

Table 6.5: Evaluation plan for recommended requirements.

Requirement	Expected behaviour	Testing method
Natural-language explanation using an existing model	LLM explains backend-calculated results.	Prompt/context and qualitative consistency checks.
Investor profile	Users complete investor profile fields.	Controller, service and frontend checks.
Risk assessment	Profile inputs map to a risk profile.	Equivalence, boundary and branch tests.
Target allocation recommendation	Risk profiles map to target allocations.	Target allocation unit tests.
User allocation comparison against target allocation	Actual allocation is compared with target allocation.	Suitability analysis tests.
Assistant insights	Assistant insights explain portfolio/profile alignment.	Assistant branch tests and frontend evidence.
API quota reduction and caching	Repeated external requests are reduced.	Quota regression and integration tests.
New-user onboarding	New users are guided toward profile and activity setup.	Model advice tests and frontend evidence.

6.3.3 Optional Requirements

Table 6.6: Evaluation plan for optional requirements.

Requirement	Expected behaviour	Testing method
-------------	--------------------	----------------

PDF export	Available analysis results export to PDF.	E2E download check and Appendix B visual inspection.
Multiple optimisation algorithms	MVO and simplified Risk Parity are supported.	Algorithm tests and E2E algorithm selection.
Natural-language explanation using a self-developed model	Future alternative to external LLM.	Not implemented; future work.
Comparison of different NLP or LLM models	Future comparison of explanation models.	Not implemented; future work.
Feedback page	Future user feedback collection.	Not implemented; future work.

6.4 Automated Unit Testing

Automated unit tests evaluated deterministic logic in isolation from the browser and external services, especially the activity-driven calculation pipeline, assistant rules and optimisation algorithms.

6.4.1 Holdings Calculation Tests

Holdings calculation is a core part of the system because activities are treated as the source of truth. The tests therefore checked whether BUY and SELL activities are aggregated correctly and whether derived metrics are updated consistently. The holdings tests include equivalence partitioning, boundary value analysis, branch testing, loop testing and regression tests for optimisation eligibility.

6.4.2 Allocation Analysis Tests

Allocation tests evaluated whether derived holdings are correctly grouped into portfolio categories. Category partitioning was especially relevant because allocation is not only calculated by individual holding, but also by asset class, sector, market and currency. The tests covered grouping by holding, asset class, sector, market and currency, as well as missing metadata and zero-value cases.

6.4.3 Financial Assistant Tests

Financial assistant tests evaluated the rule-based assistant layer. This included risk assessment, target allocation, asset-class mapping, suitability comparison and assistant insight generation. These tests are important because the assistant is one of the main ways the final system moves beyond passive portfolio tracking. The tests covered profile classification, target allocation mapping, suitability comparison and missing-data insight branches.

6.4.4 Portfolio Optimisation Tests

Portfolio optimisation was tested at service and algorithm level, covering input preparation, eligible assets, invalid data, MVO, Risk Parity and portfolio statistics.

MVO required particular attention because its initial implementation was developed with generative AI assistance, so tests checked output structure, metrics, efficient frontier points and invalid input handling.

Risk Parity was tested as the second optimisation algorithm, checking inverse-volatility weighting, weight normalisation, missing returns and zero-volatility cases.

6.5 Controller and API Testing

Controller and API tests checked whether backend endpoints expose service logic correctly to the frontend. The tests covered assistant, investor profile, allocation, analysis, asset search and holdings controllers, focusing on request handling, validation, status codes and response structures.

6.6 Integration and Regression Testing

Integration tests covered interactions between components, while regression tests checked previously identified issues such as market-data failure, quota handling, benchmark comparison and optimisation eligibility.

Table 6.12: Integration and regression test summary.

Test file	Purpose	Result
`MarketDataFailureIntegrationTest.java`	Checks that market-data failure scenarios are handled without uncontrolled application failure.	Passed
`MarketDataQuotaRegressionTest.java`	Checks behaviour related to external API quota or repeated request risks.	Passed
`PortfolioWorkflowIntegrationTest.java`	Checks the interaction between portfolio workflow components such as activities, holdings and downstream analysis.	Passed
`PortfolioOptimisationEligibilityRegressionTest.java`	Checks that optimisation eligibility handling remains correct for supported and unsupported assets.	Passed
`BenchmarkComparisonRegressionTest.java`	Checks previously identified benchmark comparison behaviour.	Passed
`HoldingServiceOptimisationEligibilityRegressionTest.java`	Checks holding-level optimisation support and eligibility decisions.	Passed
`PortfolioOptimisationQuotaRegressionTest.java`	Checks optimisation request behaviour linked to quota or repeated market-data calls.	Passed
HoldingService regression checks	Checks that when a live latest price is unavailable, average cost is used as a fallback valuation price while avoiding repeated per-symbol single-quote fallback requests from HoldingService.	Passed

6.7 Automated End-to-End Testing

End-to-end testing was represented by one automated Playwright browser test against the locally running Spring Boot application on `localhost:8080`.

The main E2E test, `src/test/java/com/tsh11/fypcode/e2e/PortfolioMainUserJourneyE2ETest.java`, uses the populated demo account and verifies the main workflow, including PDF download trigger. It does not inspect PDF contents; these are checked through Appendix B.

Table 6.13: Automated Playwright E2E scenario.

Step	Action	Expected result	Result
------	--------	-----------------	--------

1	Open the login page.	Login form is displayed.	Passed
2	Log in using the populated test account.	User is authenticated and can access portfolio pages.	Passed
3	Navigate to holdings page.	Holdings page loads successfully.	Passed
4	Navigate to allocation page.	Allocation page loads successfully.	Passed
5	Navigate to analysis page.	Analysis page loads successfully.	Passed
6	Run Mean-Variance Optimisation.	Optimisation result content appears.	Passed
7	Select Risk Parity and run optimisation.	Risk Parity result content appears.	Passed
8	Check efficient frontier behaviour.	Efficient Frontier card is hidden for Risk Parity.	Passed
9	Click Export PDF.	PDF download is triggered.	Passed

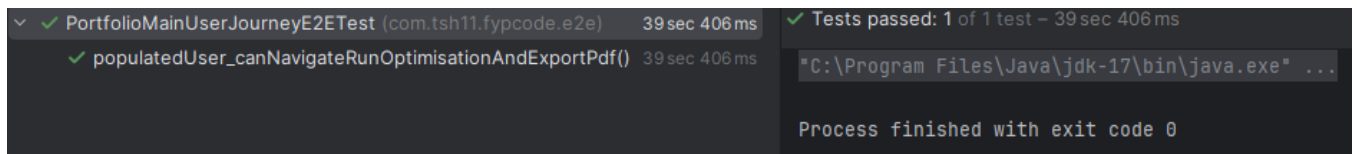


Figure 6.2: Automated Playwright E2E test result showing PortfolioMainUserJourneyE2ETest passed.

6.8 Coverage and Test Result Summary

All 216 automated tests passed. The final IntelliJ coverage report is included in Appendix C and was generated against the final codebase after unused earlier-version files were removed.

Table 6.14: Overall IntelliJ coverage summary.

Coverage metric	Result
Class coverage	61.1% (66/108)
Method coverage	61.5% (584/950)
Branch coverage	45.6% (499/1095)
Line coverage	57.6% (1684/2923)

The overall percentages are useful, but the package-level results provide more meaningful evidence for the project's main logic. The `service.assistant` package reached 98.6% line coverage and 90.6% branch coverage, while `service.portfolio` reached 98.0% line coverage and 82.4% branch coverage.

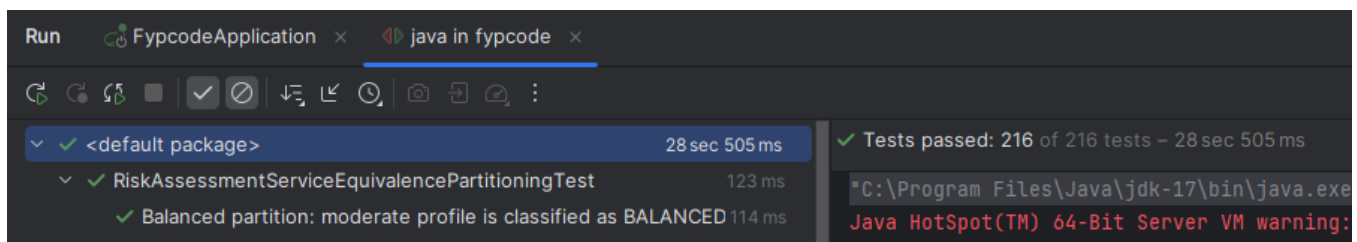


Figure 6.3: IntelliJ test result showing all 216 automated tests passed.

6.9 Evaluation of Generative AI-Assisted Component

The Mean-Variance Optimisation component required specific evaluation because its initial implementation was developed with generative AI assistance. It was not treated as correct by default and was checked through algorithm tests, optimisation service tests, E2E workflow verification and review against Chapter 3 finance concepts.

The Gemini explanation feature is also a use of generative AI, but it is used at runtime only to explain backend-calculated results. Its evaluation therefore focused on the separation between deterministic calculation and explanation, and on qualitative consistency with the calculated context.

6.10 Limitations of Testing

The testing provides evidence that the implemented system satisfies the project requirements, but it does not prove production readiness or suitability for real investment advice. The Playwright test validates one representative workflow rather than every possible frontend interaction.

Coverage results also have limitations because the report includes configuration, DTOs, integration classes and framework-driven code. The strongest coverage was achieved in assistant and portfolio optimisation packages, but high coverage does not guarantee all financial or usability issues are identified.

The LLM explanation feature was checked for consistency with backend-calculated results, but a formal user study of explanation quality was outside the project scope. The Playwright test confirmed that PDF export triggered a download, but it did not inspect the internal text or layout of the exported PDF. The exported report content was instead checked through visual inspection and supported by the sample report included in Appendix B.

6.11 Summary

This chapter evaluated the final system using automated unit tests, controller/API tests, integration tests, regression tests, automated browser-level E2E testing and coverage inspection. All 216 automated tests passed. The testing structure covers the main project areas, including holdings calculation, allocation analysis, financial assistant logic, benchmark comparison, metadata handling, optimisation services, Mean-Variance Optimisation, simplified Risk Parity and the main browser workflow.

7. Results and Discussion

This chapter discusses the result of the completed financial portfolio assistant. Chapters 5 and 6 showed how the system was implemented and tested. This chapter interprets those results by considering whether the project aim and requirements were achieved, what the main system outcomes were, how the final system compares with the background review, and what limitations remain.

7.1 Overview

The final system is a full-stack portfolio assistant for beginner and non-specialist retail investors. It allows users to record simulated investment activities, derive holdings, analyse allocation, compare performance with a benchmark, calculate optimisation scenarios, receive profile-based assistant insights, request natural-language explanations and export analysis results as a PDF report.

The main result is the integration of these functions into one coherent workflow. Investment activities act as the source of truth. Holdings are derived from those activities, and downstream features such as allocation analysis, benchmark comparison, optimisation, assistant insights, explanation and PDF export use the calculated portfolio state. This means the project outcome is closer to an educational portfolio assistant than a passive transaction tracker.

7.2 Achievement of Project Aim and Requirements

The project aim was to design, implement and evaluate a web-based financial portfolio assistant that helps beginner users understand portfolio structure, risk exposure, benchmark performance and optimisation results. Based on the final implementation and testing evidence, the main aim was achieved. The system supports user-specific

portfolio data, activity-driven calculation, visual analysis, optimisation, assistant insights, explanation and export.

Table 7.1: Summary of achievement against main project objectives.

Objective area	Result
Authentication and user data	Achieved
Activity-driven portfolio workflow	Achieved
Holdings and allocation analysis	Achieved
Market-data and benchmark support	Achieved with constraints
Portfolio optimisation	Achieved
Financial Consultant Assistant	Achieved
Explanation and PDF export	Achieved
Testing and evaluation	Achieved

The essential requirements were achieved. The recommended requirements were also largely achieved, especially the investor-profile assistant, natural-language explanation, onboarding support and caching-related improvements. Optional requirements were partly achieved: PDF export and multiple optimisation algorithms were implemented, while a self-developed LLM, model comparison and a feedback page remain outside the final implementation.

7.3 Discussion of Main System Results

7.3.1 Activity-Driven Portfolio Workflow

The activity-driven workflow is one of the strongest results of the project. Holdings are not maintained as separate persistent records; they are calculated from user activities. This reduces duplicated state and makes the portfolio easier to reason about. If the user edits the activity history, holdings and downstream analysis can be recalculated from the same source of truth.

This approach also made testing more meaningful. Unit and regression tests could focus on whether BUY and SELL records produce the correct derived holdings. The later regression fix for missing live prices further improved the result by using average cost as a fallback valuation price instead of displaying a misleading zero market value and -100% unrealised loss.

7.3.2 Portfolio Analysis and Visualisation

The holdings and allocation features allow the user to interpret the portfolio from several perspectives. Instead of only showing a list of transactions, the system calculates quantity, cost basis, average cost, market value and unrealised gain or loss where price data is available. Allocation visualisation then shows exposure by asset, asset class, sector, market and currency.

This supports the educational purpose of the project because beginner users can see concentration and diversification more clearly. The result is a system that presents portfolio structure rather than only raw records.

7.3.3 Benchmark Comparison and Optimisation

Benchmark comparison gives the user relative performance context. A portfolio return is easier to interpret when compared with a selected benchmark over the same period. This does not provide a complete judgement of portfolio quality, but it supports more meaningful interpretation than viewing portfolio return in isolation.

The optimisation result is also significant. Mean-Variance Optimisation provides risk-return metrics and efficient frontier output, while simplified Risk Parity provides a contrasting inverse-volatility allocation. Risk Parity demonstrates that the optimisation layer can support more than one algorithm. Both algorithms are presented as educational calculation scenarios rather than trading instructions or predictions of future performance.

7.3.4 Financial Consultant Assistant

The Financial Consultant Assistant changes the system from a passive dashboard into a more active decision-support tool. The user profile is used to calculate a risk profile, target allocation and suitability comparison. This allows the system to discuss whether the user's actual allocation appears aligned with their profile.

The assistant is useful because it connects portfolio analysis to user context. Allocation percentages are easier to interpret when compared with an educational target allocation. However, the assistant remains rule-based and educational, not regulated financial advice.

7.3.5 Natural-Language Explanation and PDF Export

The natural-language explanation feature addresses the interpretability problem identified in the early chapters. Metrics such as volatility, Sharpe ratio and optimisation weights may be technically useful but difficult for beginner users to understand. The system therefore uses Gemini to explain backend-calculated results in readable language.

A key design result is that calculation and explanation remain separated. Backend services calculate holdings, allocation, benchmark comparison, optimisation outputs and assistant insights. The LLM explains those results rather than calculating them independently. PDF export complements this by allowing the user to save available analysis results for later review, with the sample output provided in Appendix B.

7.4 Comparison with Existing Systems and Background Findings

Chapter 3 compared portfolio trackers, robo-advisors, financial analytics platforms and AI-assisted finance tools. The final system does not attempt to outperform professional products in scale or market coverage. Its value is that it combines selected ideas from these categories into a student-built educational assistant.

Table 7.2: Comparison of the final system with existing system categories.

System category	Typical focus	How this project differs
Portfolio trackers	Track transactions, holdings and values.	This project also adds benchmark comparison, optimisation, assistant insights and explanation.
Robo-advisors	Use investor profiles to suggest or manage portfolios.	This project uses profiling educationally and does not manage real money or execute trades.
Financial analytics platforms	Provide broad market data and advanced analysis.	This project is narrower but more beginner-focused and explanatory.
AI-assisted finance tools	Use AI to support finance workflows.	This project constrains AI to explanation of deterministic backend results.

This comparison supports the project's design position. The system is not a brokerage platform, professional analytics terminal or full robo-advisor. It is an educational decision-support application that combines portfolio tracking, analysis, suitability comparison and explanation in one workflow.

7.5 Strengths of the Final System

The main strength of the final system is that it connects portfolio features into a coherent activity-driven workflow rather than leaving them as isolated pages.

A second strength is separation of responsibilities: controllers, services, repositories, integrations, assistant logic and optimisation algorithms have distinct roles.

A third strength is the controlled use of generative AI. The project uses Gemini for explanation, but it keeps financial calculation in deterministic backend services. This supports the educational purpose of the system while reducing the risk of unsupported AI-generated calculations.

Finally, the project is supported by structured testing evidence, including 216 passing automated tests, regression checks, coverage inspection and a Playwright workflow test.

7.6 Challenges and Resolutions

Several challenges affected the final system. They were mainly related to external data, calculation robustness, multiple algorithm behaviour and explanation boundaries.

Table 7.3: Main implementation challenges and resolutions.

Challenge	Resolution	Result
Market-data quota and repeated requests	Caching, batching and frontend changes reduced unnecessary calls.	Reduced pressure on the external API and improved repeatability.
Missing latest price caused misleading holding value	Average cost is used as a fallback when live price is unavailable.	Avoids zero market value and misleading -100% unrealised loss.
Different optimisation algorithms produce different outputs	The frontend shows the efficient frontier for MVO and hides it for Risk Parity.	The UI now reflects the output of the selected algorithm.
LLM output may be unsupported if unconstrained	The backend sends calculated context and uses the LLM only for explanation.	Calculation remains deterministic and testable.
PDF export could trigger expensive recalculation	Export uses current page state instead of rerunning analysis automatically.	Export behaves as a reporting feature rather than a calculation feature.
Beginner users may not know the workflow	Onboarding, UI-only navigation guidance and help popovers were added.	The interface better guides users through setup and profile completion.

These challenges show that the project required both feature implementation and refinement. The final system became more robust by handling missing data, avoiding misleading values and making algorithm-specific behaviour visible to the user.

7.7 Limitations

Although the project achieved its aim, it has limitations. Financially, it is an educational decision-support tool and not regulated financial advice. It does not execute trades, connect to brokerage accounts, manage real money or guarantee investment returns. Optimisation uses historical data and simplified assumptions, so results should be interpreted as scenarios rather than predictions.

There are also technical limitations. Market data depends on Alpha Vantage availability, asset support and quota limits. Some assets may have incomplete metadata or insufficient historical data for optimisation. The simplified Risk Parity implementation uses inverse-volatility weighting and does not solve a full equal-risk-contribution model. The LLM explanation feature depends on an external provider and produces natural-language output that requires careful interpretation.

The evaluation also has limitations. The test suite provides evidence for the implemented logic and one representative browser workflow, but it does not prove production readiness. No formal user study was conducted, so the project cannot fully prove that beginner investors understand the explanations better than they would without the system. These limitations are addressed as possible future work in Chapter 8.

7.8 Summary

This chapter discussed the results of the completed project. The system achieved its main aim by implementing a working financial portfolio assistant that combines activity tracking, derived holdings, allocation analysis, benchmark comparison, selectable optimisation algorithms, investor-profile suitability analysis, natural-language explanation and PDF export.

The main contribution is the integration of these functions into an educational workflow for beginner investors. The system remains limited by its educational scope, reliance on external services, simplified financial assumptions and absence of a formal user study, but it provides a coherent and tested final software artefact. The next chapter concludes the report and outlines future work.

8. Conclusion and Future Work

8.1 Project Conclusion

This project set out to design, implement and evaluate a web-based financial portfolio assistant for beginner and non-specialist retail investors. The final system achieved this aim by combining activity-driven portfolio tracking, portfolio analysis, optimisation, investor-profile-based assistant logic, natural-language explanation and PDF export in one coherent workflow. The system is intended as an educational decision-support tool rather than a trading platform or regulated financial adviser.

The system also achieved the main analytical objectives of the project. It calculates holdings and holding metrics, visualises portfolio allocation by different categories, compares portfolio performance against benchmarks, and calculates optimisation scenarios using Mean-Variance Optimisation and simplified Risk Parity. The Financial Consultant Assistant extends the system beyond passive portfolio tracking by using investor profile information to calculate a risk profile, target allocation, suitability comparison and assistant insight. The Gemini explanation integration then converts backend-calculated results into beginner-friendly text, while keeping financial calculation separate from the LLM.

The implementation was evaluated through automated unit tests, controller/API tests, integration tests, regression tests, a Playwright browser-level end-to-end test and IntelliJ coverage inspection. The testing evidence showed that the central calculation packages were strongly covered, while regression tests addressed practical issues such as market-data quota risks, optimisation eligibility and missing latest-price valuation fallback. Overall, the final system satisfies the core aim of producing a working financial portfolio assistant that helps users record, analyse, interpret and export portfolio information.

8.2 Future Work

Some optional requirements were completed in the final implementation, including PDF export and support for multiple optimisation algorithms. The remaining optional

requirements provide suitable directions for future work. These extensions would build on the existing architecture without changing the main activity-driven portfolio model.

8.2.1 Self-Developed or Fine-Tuned Explanation Model

A future version could investigate a self-developed or fine-tuned explanation model. The current system uses Gemini through Google GenAI as an external LLM provider, while all financial calculations remain the responsibility of deterministic backend services. This was suitable for the project because the main aim was to integrate explanation into the portfolio assistant workflow rather than to train a language model.

A self-developed or adapted model could be trained specifically for portfolio explanation tasks. This could reduce dependence on an external provider and allow more control over tone and terminology, but would require a suitable dataset, evaluation and safeguards against unsupported financial statements.

8.2.2 Comparison of Different LLM or NLP Models

Another future extension is to compare different LLM or NLP models for explanation quality. The final implementation uses one LLM provider, so it does not evaluate whether alternative models would produce clearer, safer or more consistent explanations. A future study could compare models using criteria such as readability, factual consistency with calculated results, hallucination risk, response time, cost and robustness when portfolio data is incomplete.

This would make the explanation component more systematically evaluated and link explanation quality to the project's educational purpose.

8.2.3 Feedback Page and User Evaluation

A feedback page could also be added in a future version. This was an optional requirement that was not implemented in the final project, but it would be valuable because the system is aimed at beginner and non-specialist investors. Users could rate whether explanations are understandable, whether assistant insights are useful, whether onboarding guidance is clear, and whether the exported PDF report supports later review.

This feature could also support a formal user evaluation by collecting responses on explanation clarity, assistant usefulness and interface design.

8.2.4 Other Technical Extensions

Further technical extensions could include adding more market-data providers, expanding automated frontend tests, and improving optimisation with full equal-risk-contribution Risk Parity, transaction costs or user-defined constraints. Longer-term brokerage or portfolio-file import would require additional attention to data security, privacy and regulatory boundaries.

8.3 Final Summary

Overall, the project achieved its aim of producing a working financial portfolio assistant for beginner and non-specialist investors. The final system combines activity-driven portfolio tracking, allocation analysis, benchmark comparison, portfolio optimisation, investor-profile suitability analysis, natural-language explanation and PDF export. It does not replace professional financial advice or production investment systems, but it demonstrates how software engineering techniques can be used to make portfolio analysis more understandable, testable and accessible.

Reference

Portfolio theory and optimisation:

- [1] Markowitz, H. (1991) Portfolio Selection: Efficient Diversification of Investments. 2nd ed. Cambridge, MA: Blackwell.
- [2] Sharpe, W. F. (1964) Capital asset prices: A theory of market equilibrium under conditions of risk. The Journal of Finance, 19(3), pp. 425-442.
- [3] Sharpe, W. F. (1994) The Sharpe Ratio. The Journal of Portfolio Management, 21(1), pp. 49-58.
- [4] Michaud, R. O. (1989) The Markowitz Optimization Enigma: Is "Optimized" Optimal? Financial Analysts Journal, 45(1), pp. 31-42.
- [5] Roncalli, T. (2014) Introduction to Risk Parity and Budgeting. Boca Raton: CRC Press.
- [6] Page, S. (2016) Risk Parity Fundamentals. Quantitative Finance, 16(12), pp. 1801-1802.

Data visualisation and charts:

- [7] Tufte, E. R. (2001) The Visual Display of Quantitative Information. 2nd ed. Cheshire, CT: Graphics Press.

LLM, transformers, and data-to-text explanation:

- [8] Vaswani, A. et al. (2017) Attention is all you need. Advances in Neural Information Processing Systems, 30.
- [9] Gatt, A. and Krahmer, E. (2018) Survey of the State of the Art in Natural Language Generation: Core tasks, applications and evaluation. Journal of Artificial Intelligence Research, 61, pp. 65-170.
- [10] Yang, H. et al. (2024) FinRobot: An open-source AI agent platform for financial applications using large language models. arXiv preprint arXiv:2405.14767.
- [11] Wu, S. et al. (2023) BloombergGPT: A Large Language Model for Finance. arXiv preprint arXiv:2303.17564.
- [12] Sai, A. B. et al. (2023) A Survey of Evaluation Metrics Used for NLG Systems. ACM Computing Surveys, 55(2), pp. 1-39.

Existing products and systems:

- [13] Moneyfarm (no date) Moneyfarm UK. Available at: <https://www.moneyfarm.com/uk/>
- [14] Ghostfolio (no date) Ghostfolio GitHub repository. Available at: <https://github.com/ghostfolio/ghostfolio>
- [15] OpenBB (no date) OpenBB GitHub repository. Available at: <https://github.com/OpenBB-finance/OpenBB>

Market data:

- [16] Alpha Vantage (no date) Alpha Vantage API Documentation. Available at: <https://www.alphavantage.co/documentation/>

Technology used:

- [17] Spring (no date) Spring Boot Documentation. Available at: <https://docs.spring.io/spring-boot/documentation.html>
- [18] Chart.js (no date) Chart.js Documentation. Available at: <https://www.chartjs.org/docs/latest/>

Software testing methodology:

- [19] Ammann, P. and Offutt, J. (2017) Introduction to Software Testing. 2nd ed. Cambridge: Cambridge University Press.

Others:

- [20] Kuzniak, S. et al. (2015) The Grable and Lytton risk-tolerance scale: A 15-year retrospective. Financial Services Review, 24(2), pp. 177-192.

[21] Ferson, W. E. (2010) Investment performance evaluation. Annual Review of Financial Economics, 2(1), pp. 207-234.

[22] Financial Conduct Authority (no date) FCA Handbook. Available at: <https://handbook.fca.org.uk/home>

[23] Zhang, W. and Zhang, J. (2025) Hallucination mitigation for retrieval-augmented large language models: A review. Mathematics, 13(5), 856.

[24] PostgreSQL: <https://www.postgresql.org/docs/>

[25] Google Gemini API: <https://ai.google.dev/gemini-api/docs>

[26] Playwright: <https://playwright.dev/>

Appendices

A. Database Schema

Table A.1: Database table summary

Table	Purpose
app_users	Stores registered user account details used for authentication and user-specific portfolio data.
activities	Stores simulated investment activities such as BUY and SELL records. These records are the source of truth for derived holdings.
asset_profiles	Stores asset metadata such as asset class, market, currency, provider support flags, and optimisation support.
investor_profiles	Stores investor questionnaire data, risk score, and risk profile used by the Financial Consultant Assistant.

Table A.2: app_users table

Column	Type	Notes
id	uuid	Primary key.
created_at	timestamp(6) with time zone	Account creation timestamp.
display_name	character varying(120)	User display name.
email	character varying(120)	User email address. Unique constraint.
password_hash	character varying(255)	Hashed password.
role	character varying(40)	User role.

Primary key: app_users_pkey on id. **Unique constraint:** email.

Table A.3: activities table

Column	Type	Notes
id	uuid	Primary key.
comment	character varying(500)	Optional activity comment.
currency	character varying(3)	Activity currency.
date	timestamp(6) with time zone	Activity date/time.
fee	numeric(19,6)	Optional transaction fee.
quantity	numeric(19,6)	Activity quantity.
symbol	character varying(50)	Asset symbol.
type	character varying(40)	Activity type.
unit_price	numeric(19,6)	Unit price for the activity.
user_id	uuid	Links the activity to the user.
asset_type	character varying(40)	Asset type classification.
display_name	character varying(150)	Asset display name.
address	character varying(255)	Optional address field for manual or real-estate style assets.

Primary key: activities_pkey on id. **Check constraints:** type is restricted to BUY, SELL, FEE, CASH_DEPOSIT, CASH_WITHDRAWAL and MANUAL_ASSET_UPDATE; asset_type is restricted to supported asset categories.

Table A.4: asset_profiles table

Column	Type	Notes
id	uuid	Primary key.
asset_class	character varying(40)	Asset class such as equity, ETF, bond, cash, crypto or other.
created_at	timestamp(6) with time zone	Creation timestamp.
currency	character varying(3)	Asset currency.
data_source	character varying(40)	Data source such as Alpha Vantage or manual.
display_name	character varying(150)	Asset display name.
manual_valuation_required	boolean	Indicates whether manual valuation is required.
market	character varying(100)	Market or exchange information.
price_data_supported	boolean	Indicates whether price data is supported.
sector	character varying(100)	Sector classification.
symbol	character varying(50)	Asset symbol.
updated_at	timestamp(6) with time zone	Last update timestamp.
user_id	uuid	Links the asset profile to the user.
provider_asset_type	character varying(100)	Provider-specific asset type.
quote_supported	boolean	Indicates whether quote data is supported. Default: true.
historical_price_supported	boolean	Indicates whether historical prices are supported. Default: false.
optimisation_supported	boolean	Indicates whether the asset can be used in optimisation. Default: false.

Primary key: asset_profiles_pkey on id. **Unique constraint:** user_id and symbol. **Check constraints:** asset_class and data_source are restricted to supported values.

Table A.5: investor_profiles table

Column	Type	Notes
id	uuid	Primary key.
age	integer	User age.
created_at	timestamp(6) with time zone	Creation timestamp.
income_stability	character varying(40)	Income stability level.
investment_experience	character varying(40)	Investment experience level.
investment_goal	character varying(40)	Investment goal.
investment_horizon_years	integer	Investment horizon in years.
risk_profile_type	character varying(40)	Calculated risk profile.
risk_score	integer	Calculated risk score.
risk_tolerance	character varying(40)	User risk tolerance.
updated_at	timestamp(6) with time zone	Last update timestamp.
user_id	uuid	Links the investor profile to the user.

Primary key: investor_profiles_pkey on id. **Unique constraint:** user_id. **Check constraints:** income_stability, investment_experience, investment_goal, risk_profile_type and risk_tolerance are restricted to supported enum-like values.

B. Sample Exported PDF Report

Portfolio Analysis Report

Generated on: 27/05/2026, 15:35:05
Benchmark: SPX
Benchmark period: 2025-05-27 to 2026-05-27
Optimisation period: 2025-05-27 to 2026-05-27
Optimisation algorithm: RISK_PARITY

Portfolio Summary

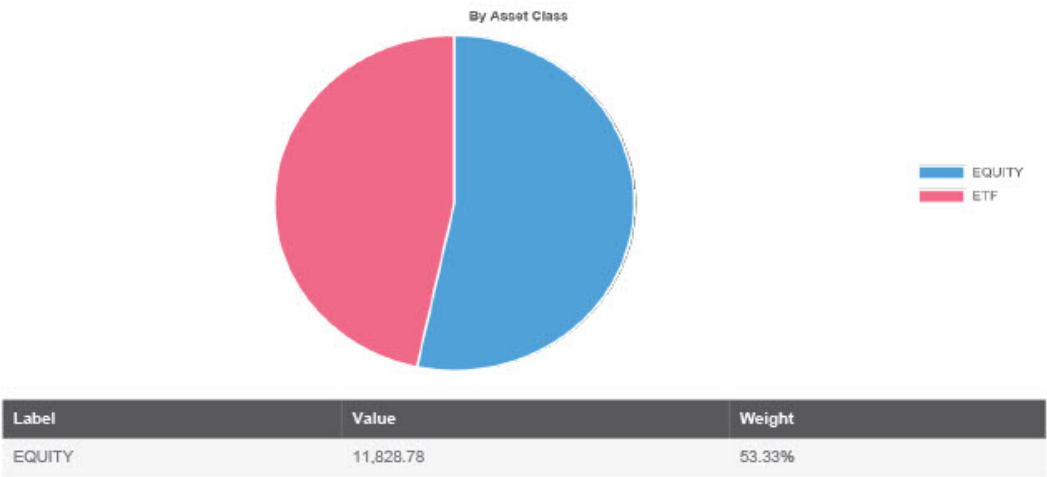
Metric	Value
Number of holdings	8
Total cost	13,209.00
Current market value	22,180.02
Unrealised gain/loss	8,971.02
Unrealised gain/loss %	67.92%

Holdings

Symbol	Name	Class	Qty	Avg Cost	Latest	Value	P/L	Currency
ADBE	Adobe Inc	EQUITY	20	14.50	240.49	4,809.80	4,519.80	USD
BND	VANGUARD TOTAL BOND MARKET INDEX FUND ETF SHARES	ETF	30	72.03	73.21	2,196.30	35.30	USD
JNJ	Johnson & Johnson	EQUITY	10	155.15	230.18	2,301.80	750.30	USD
MSFT	Microsoft Corporation	EQUITY	6	350.25	416.03	2,496.18	394.68	USD
QQQ	Invesco QQQ Trust Series 1	ETF	5	430.30	730.28	3,651.40	1,499.90	USD
SBHMY	Sino Biopharmaceutical Ltd	EQUITY	100	14.30	14.29	1,428.50	-1.50	USD
SPY	SPDR S&P 500 ETF Trust	ETF	6	520.25	750.59	4,503.54	1,382.04	USD
SVTRF	Severn Trent plc	EQUITY	20	20.10	39.63	792.50	390.50	USD

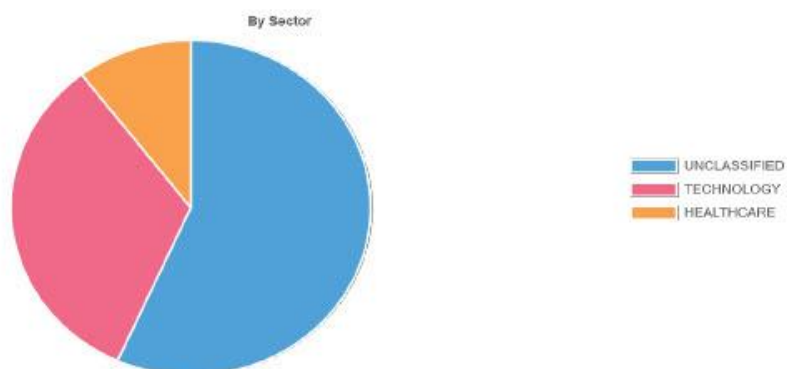
Allocation

By Asset Class



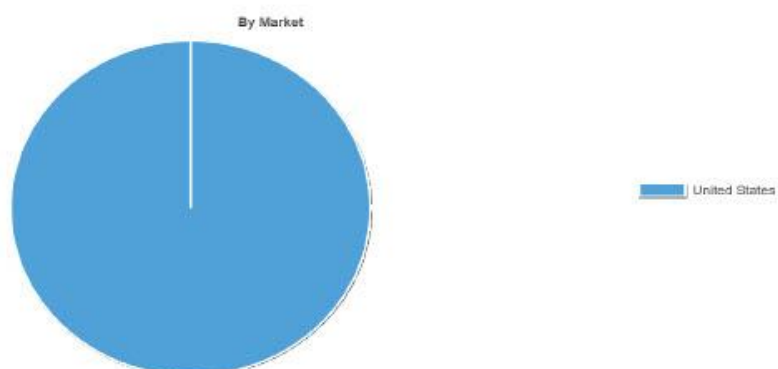
Label	Value	Weight
ETF	10,351.24	46.67%

By Sector



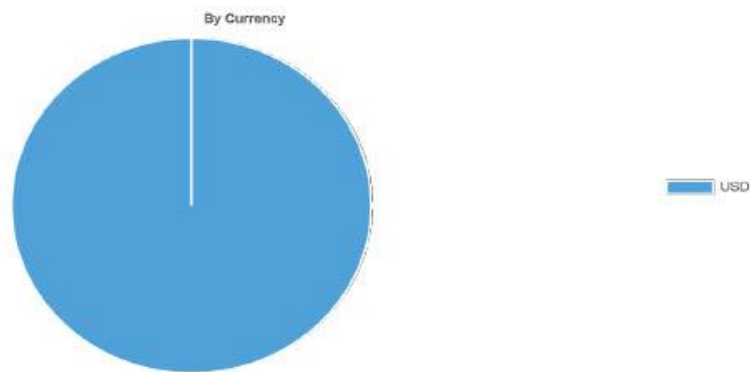
Label	Value	Weight
UNCLASSIFIED	12,572.24	56.68%
TECHNOLOGY	7,305.98	32.94%
HEALTHCARE	2,301.80	10.38%

By Market



Label	Value	Weight
United States	22,180.02	100.00%

By Currency



Label	Value	Weight
USD	22,180.02	100.00%

Benchmark Comparison

Metric	Value
Benchmark	SPX
Date range	12 Jan 2026 to 26 May 2026
Portfolio return	433.25%
Portfolio value	Start 4,159.44 ! End 22,180.02
Benchmark return	7.77%
Benchmark value	Start 6,977.32 ! End 7,519.12
Excess return	425.48%
Result	Your portfolio outperformed SPX over this period.

Indexed Performance Chart



Portfolio Optimisation

Metric	Value
Current return	9.35%
Current volatility	11.56%
Current Sharpe ratio	0.6357
Optimised return	14.38%
Optimised volatility	6.27%
Optimised Sharpe ratio	1.9729
Sharpe improvement	Sharpe ratio improved by 1.3372

Allocation Comparison

Asset	Current Weight	Target Weight	Difference	Suggested Change
ADBE	21.89%	5.14%	-16.55%	- 15.26 units (3,670.44)
BND	9.90%	43.53%	+33.63%	+ 101.89 units (7,459.59)
JNJ	10.38%	10.00%	-0.38%	- 0.36 units (83.70)
MSFT	11.25%	6.98%	-4.27%	- 2.28 units (948.06)
QQQ	16.46%	10.33%	-6.13%	- 1.86 units (1,359.62)
SBHMY	6.44%	3.13%	-3.31%	- 51.33 units (733.19)
SPY	20.30%	13.94%	-6.36%	- 1.88 units (1,410.68)
SVTRF	3.57%	6.94%	+3.36%	+ 18.83 units (746.10)

Suggestions

- Reduce allocation to ADBE
- Increase allocation to BND
- Reduce allocation to MSFT
- Reduce allocation to QQQ
- Reduce allocation to SBHMY
- Reduce allocation to SPY
- Increase allocation to SVTRF

Warnings

- No warnings.

Explanation

Portfolio Explanation

The calculation suggests a way to get more return for less ups and downs in your portfolio.

****Your Investments Now vs. The New Suggestion****

Right now, your current set of investments (your portfolio) has an estimated possible yearly gain (expected yearly return) of 9.3%. The value of these investments may move up and down (volatility) by about 11.6% each year. A number called the Sharpe ratio, which tells you how much return you get for the amount of ups and downs, is 0.64 for your current portfolio.

The calculation suggests a different way to arrange your investments. With this new mix, the estimated possible yearly gain (expected yearly return) could be 14.4%. This is a gain of 5% more per year compared to your current portfolio. Also, the suggested mix might move up and down (volatility) by only 6.3%. This is 5.3% less ups and downs than before. The new Sharpe ratio is much higher at 1.97, meaning a lot more return for the amount of movement in value. The calculation also found that your current set of investments performed better than the SPX benchmark over the same period.

****What Changes The Calculation Suggests****

To reach this suggested new mix, the calculation recommends changing the percentage of your money in certain investments. The biggest suggested change is to greatly increase your money in BND. Currently, 9.9% of your portfolio is in BND, and the system suggests raising this to 43.5%. This is an increase of 33.6%. The calculation also suggests reducing your money in ADBE significantly, from 21.7% down to 5.1%, which is a drop of 16.5%. You are also suggested to put less money into MSFT, QQQ, SBHMY, and SPY, and a little more into SVTRF.

****Important Notes****

There are no warnings from the system about these suggested changes.

This is an explanation of the calculation, not financial advice.

Disclaimer

This report is for educational guidance only. It is based on user-provided portfolio data, available market data, and algorithmic calculations. It does not constitute regulated financial advice.

[C. IntelliJ Coverage Report](#)

Report location: fypcode\htmlReport\index.html

Current scope: all classes

Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %
all classes	61.1% (66/108)	61.5% (584/950)	45.6% (499/1095)	57.6% (1684/2923)

Coverage Breakdown

Package	Class, %	Method, %	Branch, %	Line, %
com.tsh11.fypcode.service.portfolio	100% (6/6)	98.3% (58/59)	82.4% (117/142)	98% (341/348)
com.tsh11.fypcode.service.assistant	100% (10/10)	100% (47/47)	90.6% (126/139)	98.6% (280/284)
com.tsh11.fypcode.domain.investor	100% (8/8)	92.1% (35/38)		92.3% (60/65)
com.tsh11.fypcode.domain.asset	100% (4/4)	90% (36/40)		91.3% (63/69)
com.tsh11.fypcode.domain.activity	100% (2/2)	64.3% (18/28)		70.6% (24/34)
com.tsh11.fypcode.dto	87.5% (7/8)	97.3% (72/74)		91.7% (77/84)
com.tsh11.fypcode.dto.response	72.2% (13/18)	71% (203/286)		71.8% (216/301)
com.tsh11.fypcode.security	50% (1/2)	40% (2/5)		26.7% (4/15)
com.tsh11.fypcode.controller	43.8% (7/16)	34.5% (20/58)	0% (0/12)	41.2% (56/136)
com.tsh11.fypcode.service	35.7% (5/14)	41% (50/122)	46.2% (256/554)	51.8% (518/1000)
com.tsh11.fypcode.integration.marketdata	28.6% (2/7)	37.5% (30/80)	0% (0/240)	7.8% (32/409)
com.tsh11.fypcode.dto.request	20% (1/5)	15.3% (13/85)		14.3% (13/91)
com.tsh11.fypcode.integration.llm	0% (0/1)	0% (0/2)	0% (0/6)	0% (0/8)
com.tsh11.fypcode.exception	0% (0/2)	0% (0/3)		0% (0/3)
com.tsh11.fypcode.domain.user	0% (0/1)	0% (0/11)	0% (0/2)	0% (0/14)
com.tsh11.fypcode.config	0% (0/3)	0% (0/10)		0% (0/60)
com.tsh11.fypcode	0% (0/1)	0% (0/2)		0% (0/2)

generated on 2026-05-25 21:30